



线性回归

Linear Regression

 课程名称：机器学习

 章节：第4章 线性回归

 授课人：魏军

监督学习概念

回归 (Regression)

- 如何预测上海浦东的房价?
- 未来的股票市场走向?

标签连续

分类 (Classification)

- 身高1.85m, 体重100kg的男人穿什么尺码的T恤?
- 根据肿瘤的体积、患者的年龄来判断良性或恶性?

标签离散

监督学习分为回归和分类

“回归”的由来

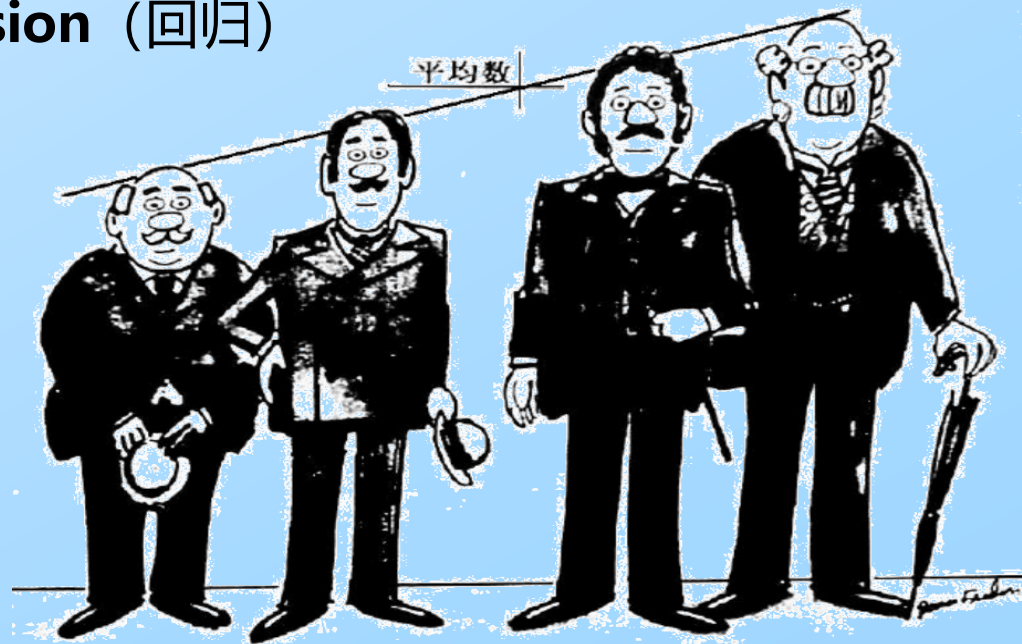
早期统计学

1870—1880 年代，英国统计学家高尔顿在研究**人类身高遗传**时

- 收集了约 **1078 对父子**的身高数据
- **极高父亲的儿子，通常比父亲矮；极矮父亲的儿子，通常比父亲高**
- 子代身高没有无限两极分化，而是**向群体平均身高“往回走”**
- 他把这种**向平均值倒退、往回靠**的趋势，命名为 **regression**（回归）

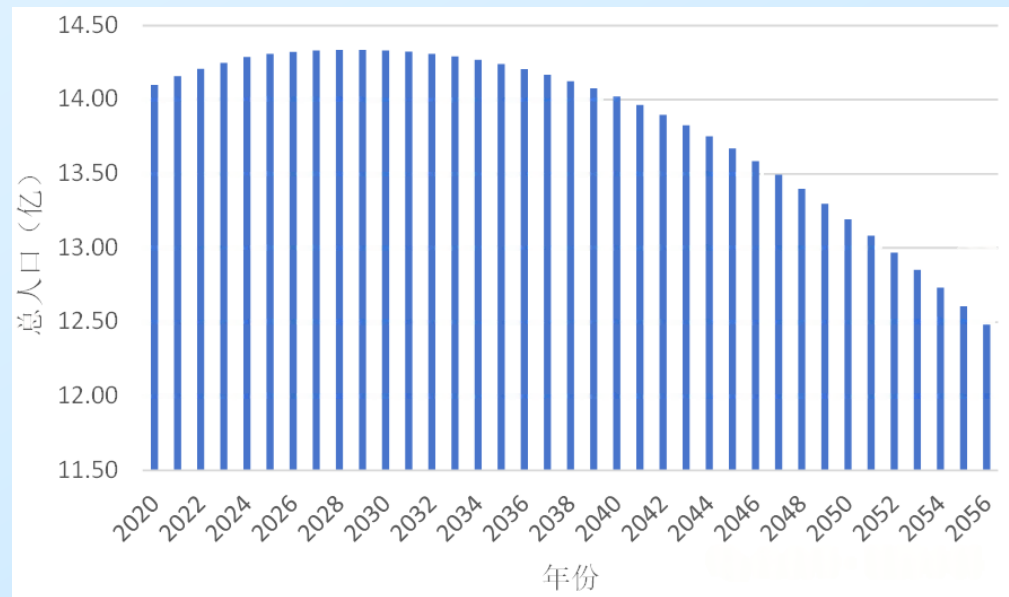
现代统计学/机器学习

- **回归 = 建模预测连续值**，与“往回、均值”已无关
- 线性回归、逻辑回归、多项式回归等，都继承了这个历史名称



“回归”的应用

人口预测



股价走势



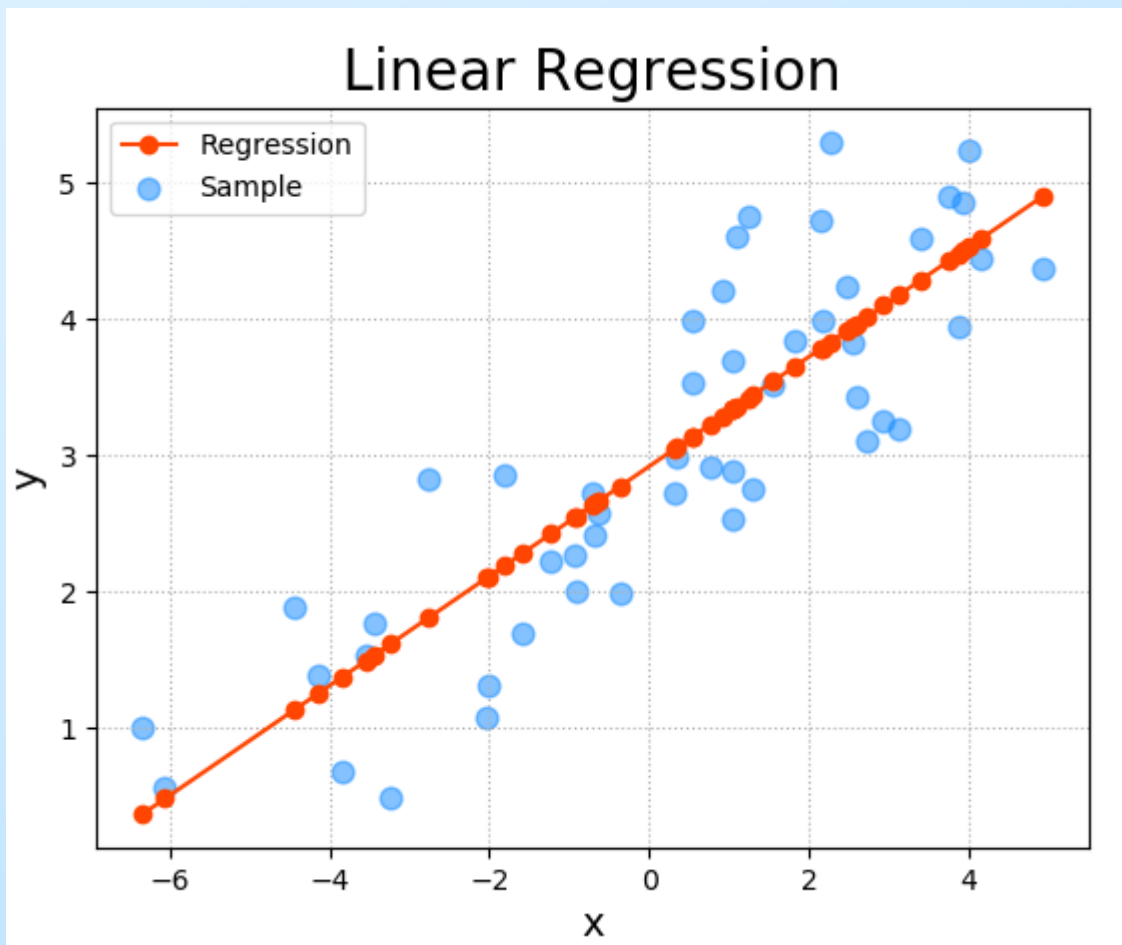
视频推荐

A screenshot of the Tencent Video (腾讯视频) homepage. The interface includes a top navigation bar with the logo, a search bar, and various icons. Below the navigation bar, there are several video recommendations, including 'JADE DYNASTY', '青云志2', '没有秘密的你', '小猪佩奇全集', '谍战深海之惊蛰', and '明月照我心'. On the right side, there is a sidebar with sections like '猜你在追' (Guess what you're watching) and '重磅推荐' (Heavy recommendation), listing various shows and movies.

线性回归原理

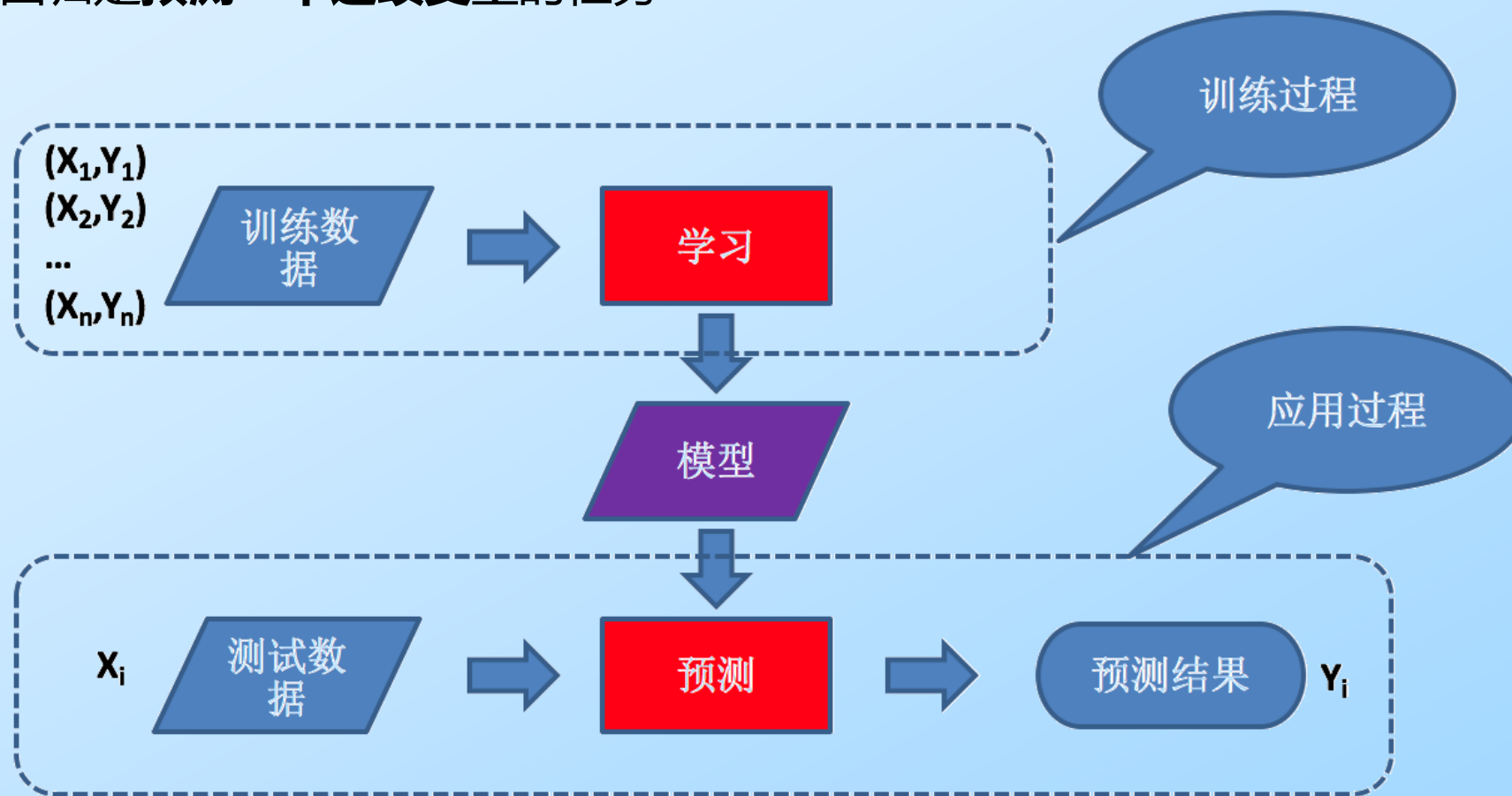
线性回归的形式

- 通过属性的线性组合来进行预测的**线性模型**
- 找到一条直线或者一个平面或者更高维的超平面，**使得预测值与真实值之间的误差最小化**



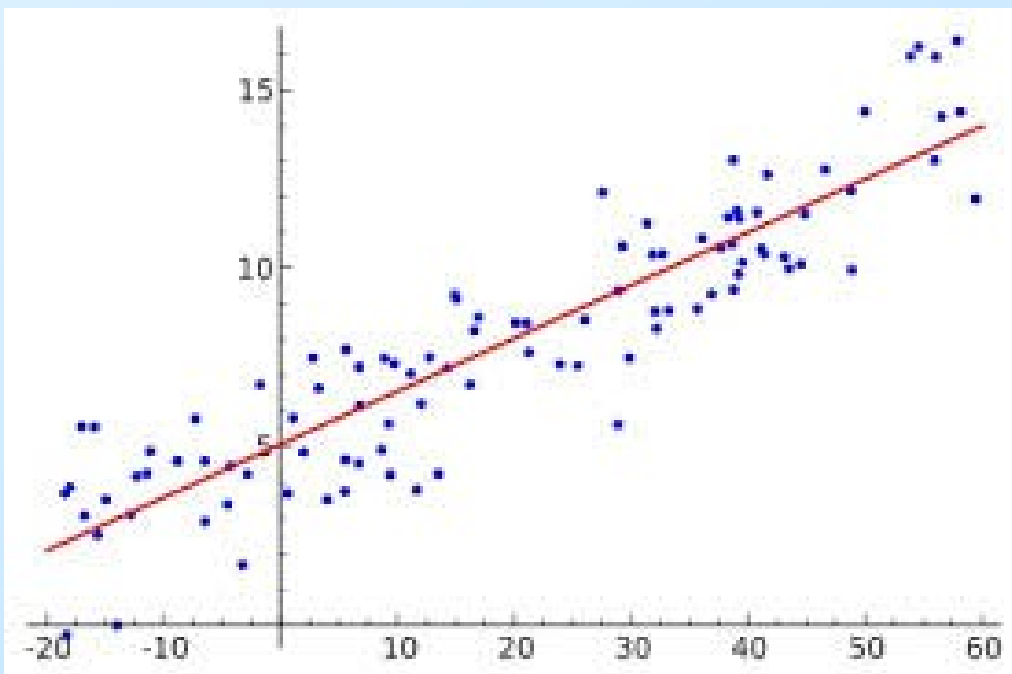
回归分析(regression analysis)

- 确定两种或两种以上变量间**相互依赖**的定量关系的一种**统计分析方法**
- 例如研究一组自变量 $(x_1, x_2, \dots, x_m)$ 和另一组因变量 $(y_1, y_2, \dots, y_m)$ 之间关系
- 回归是**预测一个连续变量**的任务

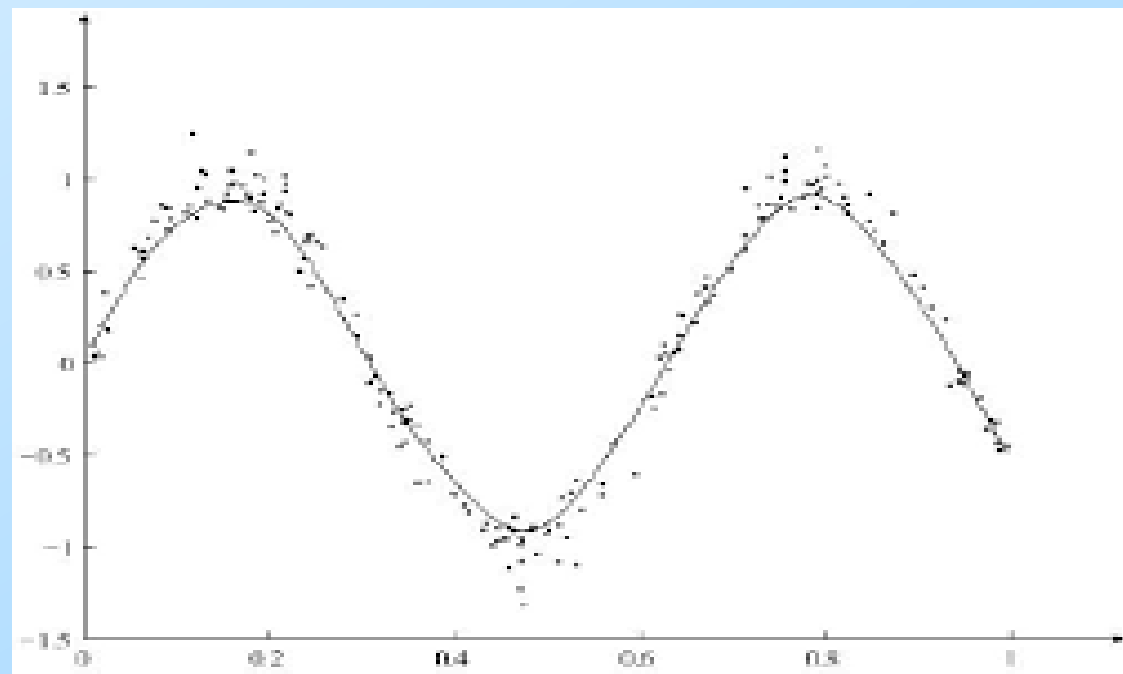


线性回归 VS 非线性回归

- 回归问题按照输入变量的个数，分为**一元回归**和**多元回归**
- 按照输入变量和输出变量之间关系，分为**线性回归**和**非线性回归**

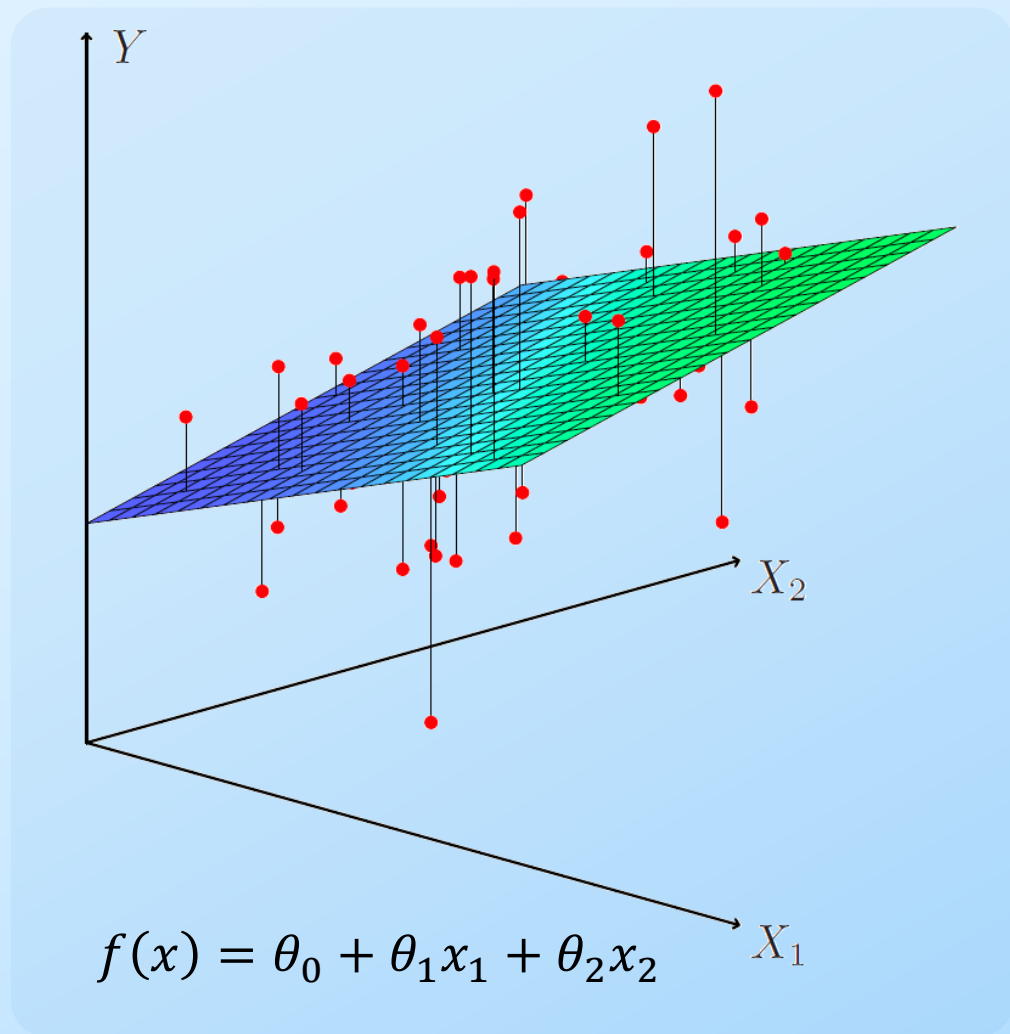


一元线性回归

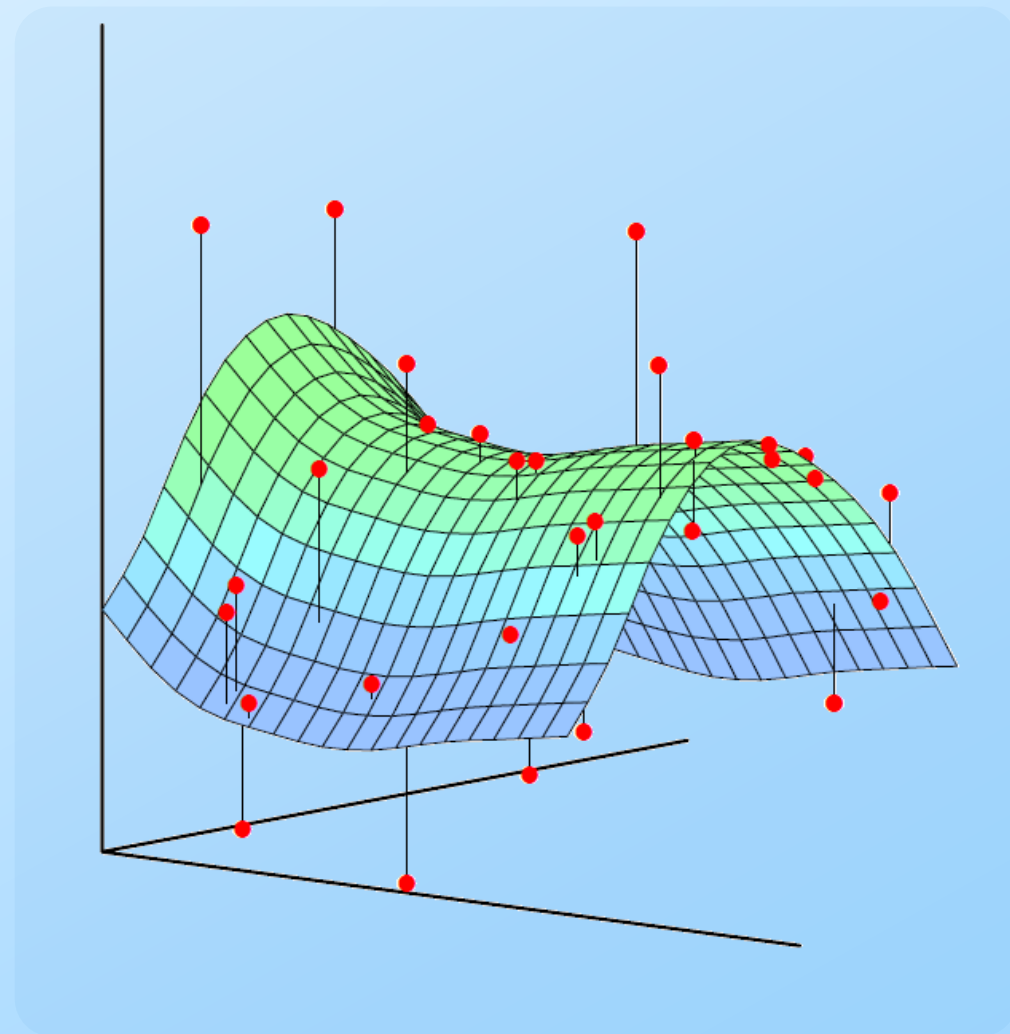


一元非线性回归

线性回归 VS 非线性回归



二元线性回归



二元非线性回归

线性回归模型

给定样本集 $D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$, 其中 $x^{(i)} = (1, x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)})^T$, m 表示样本个数, n 表示特征维数

线性回归试图学得一个线性函数以尽可能准确地预测 y , 假设线性函数如下

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

函数 $h_{\theta}(x)$ 表示以 θ 为参数, $\theta_0, \theta_1, \theta_2, \dots, \theta_n$ 是回归参数 (待求解), 上式可改写成

$$h_{\theta}(x) = \sum_{j=0}^n \theta_j x_j = \theta^T x$$

其中

$$x = \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad \theta = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{pmatrix}$$

损失函数

假设已给定了训练数据集，该如何求出参数 θ ，使预测值 $h_{\theta}(x)$ 和真实值 y 的距离越近越好

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m \mathcal{L}(h_{\theta}(x^{(i)}), y^{(i)})$$

通常 $\mathcal{L}$ 采用平方误差 (Square Error)

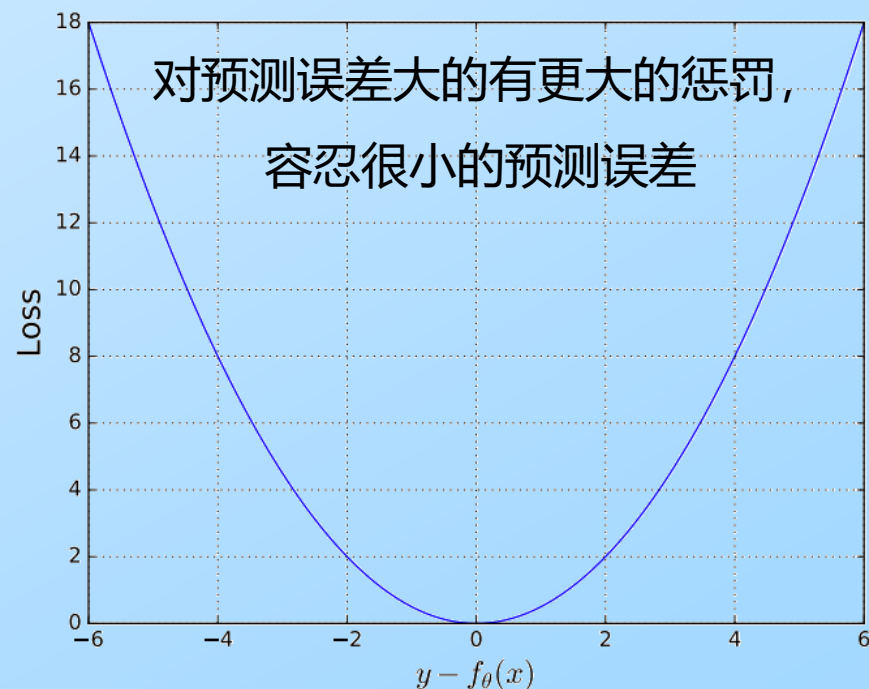
$$\mathcal{L}(y^{(i)}, h_{\theta}(x^{(i)})) = \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

最终

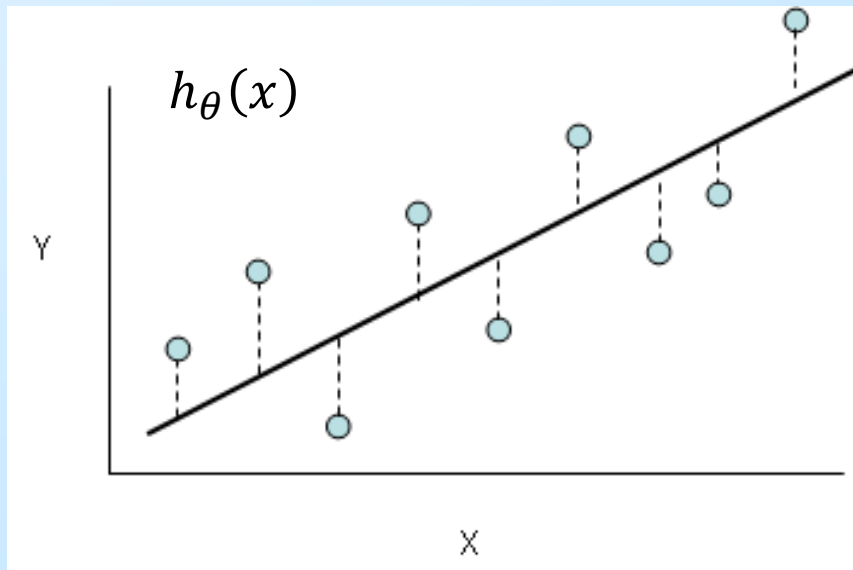
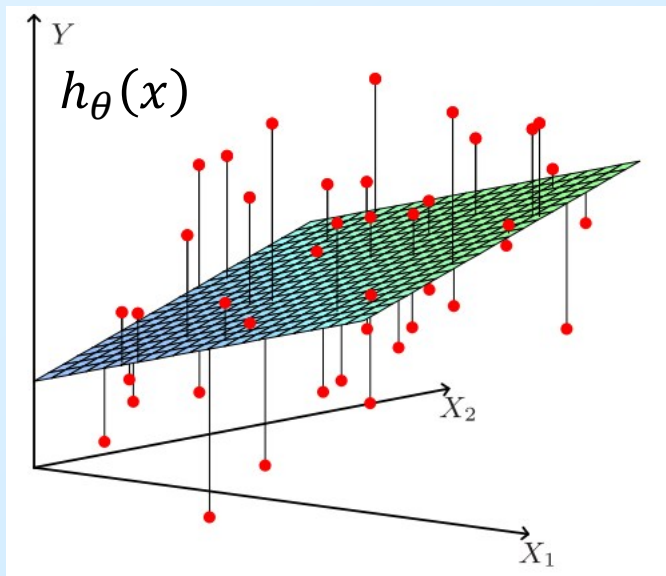
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

其中， $1/2$ 的作用在于求导时，消掉常数系数

优化目标：通过调整 θ 来让损失函数 $J(\theta)$ 取得最小值，常见的方法包括**梯度下降法**、**最小二乘法**

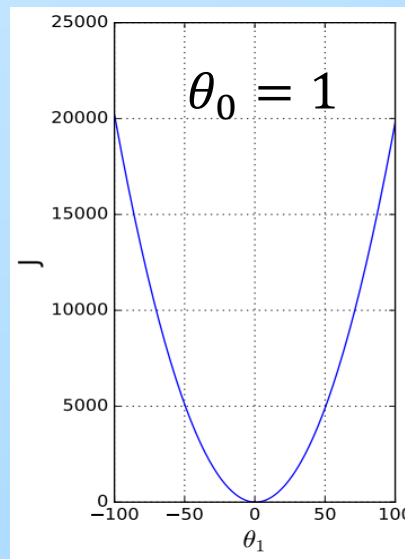
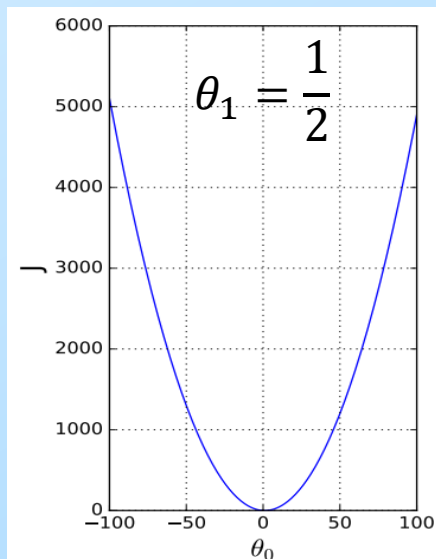
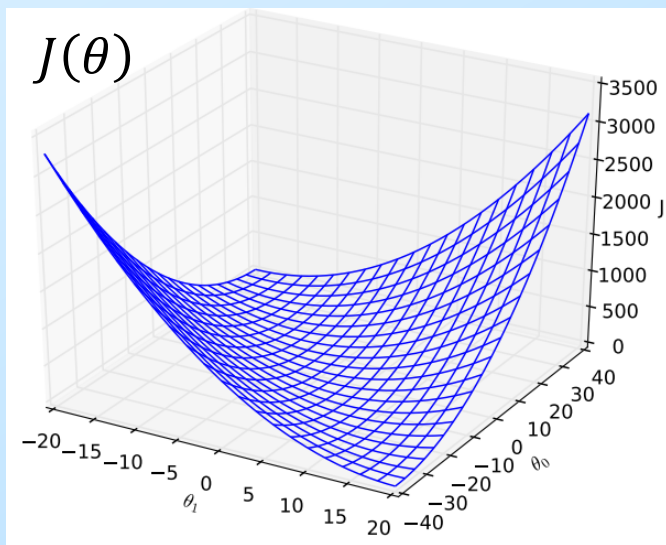


损失函数



$$\min_{\theta} J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- 假设 $h_{\theta}(x) = \theta_0 + \theta_1 x$
- 只有一个样本 $x = 2, y = 1$
- 计算 $J(\theta) = \frac{1}{2} (\theta_0 + 2\theta_1 - 1)^2$



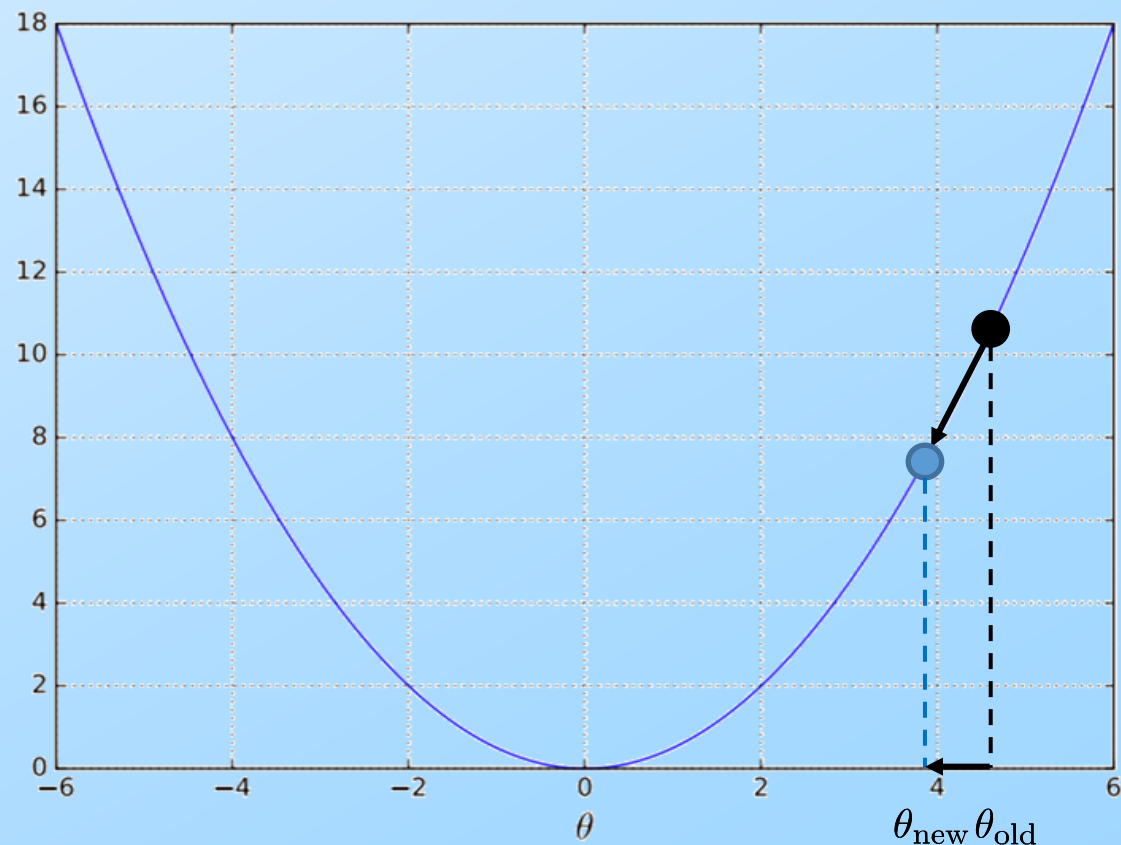
- $h_{\theta}(x)$ 是 x 的函数
- $J(\theta)$ 是 θ 的函数

线性回归求解

梯度下降法

- ✓ 对于梯度，沿着梯度向量的方向，更容易找到函数的最大值。反过来说，沿着梯度向量相反的方向，也就是 $-\frac{\partial J(\theta)}{\partial \theta}$ 的方向，梯度减少最快，更容易找到函数的最小值
- ✓ 在机器学习算法中，在最小化损失函数时，可以通过梯度下降法来一步步的迭代求解，得到最小化的损失函数和模型参数值

$$\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \alpha \frac{\partial J(\theta)}{\partial \theta}$$



梯度下降法

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

我们的目的是要误差函数尽可能的小，首先随机初始化参数，然后不断反复的更新参数使得误差函数减少，直到满足要求时停止

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} \quad \text{此时} \quad \frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

其中 α 代表**学习率**，表示每次向着J最陡峭的方向迈步的大小，线性回归中迭代公式如下：

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

常见的梯度下降法具体包含三种不同的形式（**批量梯度下降法BGD**、**随机梯度下降法SGD**、**小批量梯度下降法MBGD**），主要区别在于使用多少数据来计算目标函数的梯度。不同方法主要在准确性和优化速度间做权衡

梯度下降法

批量梯度下降法 (Batch Gradient Descent, BGD)

梯度下降的每一步中，都用到了**所有**的训练样本

优化目标 $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$

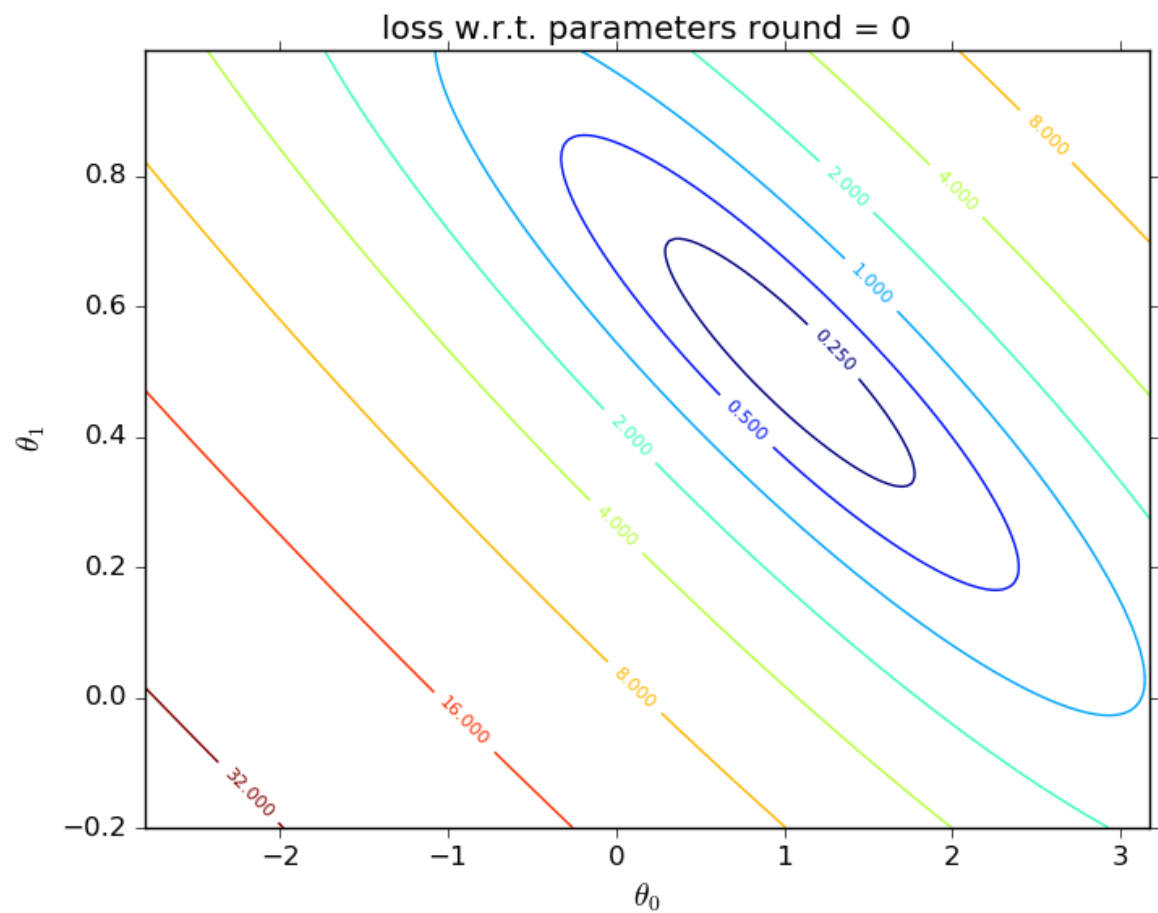
计算梯度 $\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} \sum_{i=1}^m \left((h_{\theta}(x_i) - y_i) \frac{\partial h_{\theta}(x_i)}{\partial \theta} \right) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_i$

根据整个批量数据的梯度更新参数 $\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \alpha \frac{\partial J(\theta)}{\partial \theta}$

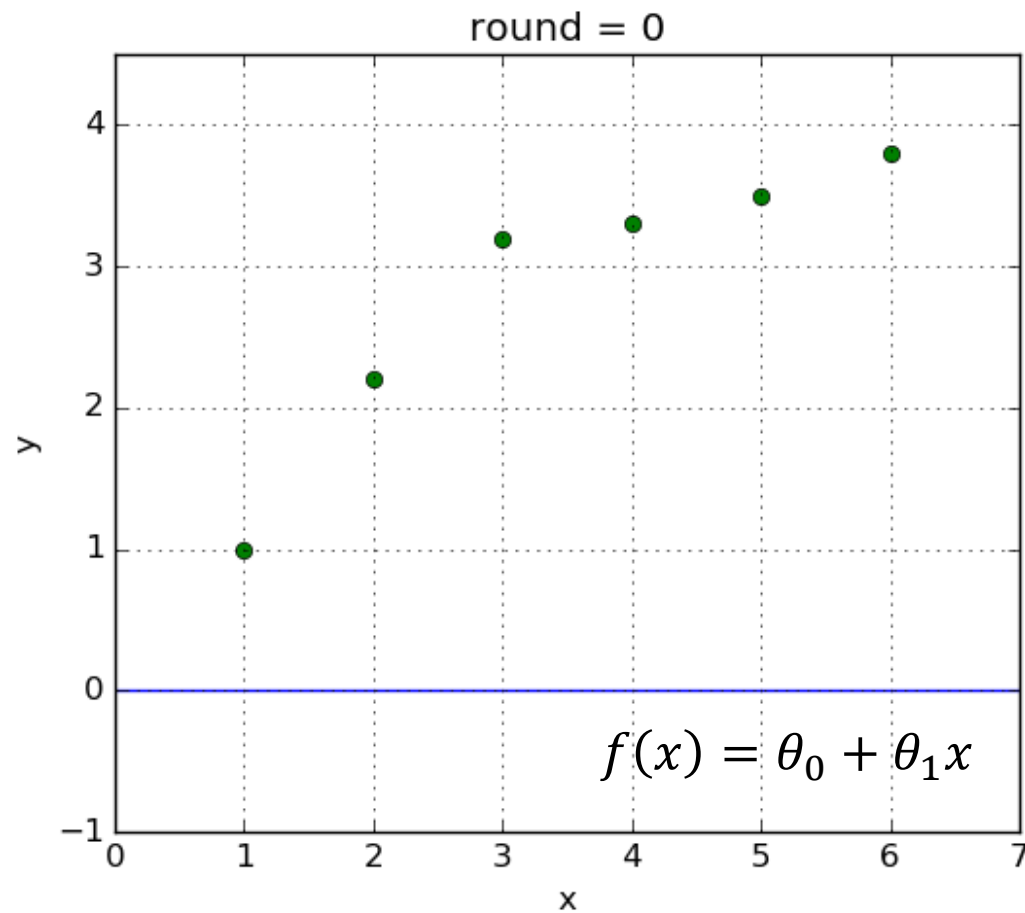
$$\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_i$$

梯度下降法

批量梯度下降法 (Batch Gradient Descent, BGD)



损失函数 $J(\theta)$ 变化



假设函数 $h_{\theta}(x)$ 变化

梯度下降法

随机梯度下降 (Stochastic Gradient Descent, SGD)

梯度下降的每一步中，只使用一个样本估算梯度，而不需要使用所有样本

优化目标
$$\min_{\theta} J^{(i)}(\theta) = \frac{1}{2} (h_{\theta}(x_i) - y_i)^2$$

计算梯度
$$\frac{\partial J^{(i)}(\theta)}{\partial \theta} = (h_{\theta}(x_i) - y_i) \frac{\partial h_{\theta}(x_i)}{\partial \theta} = (h_{\theta}(x_i) - y_i) x_i$$

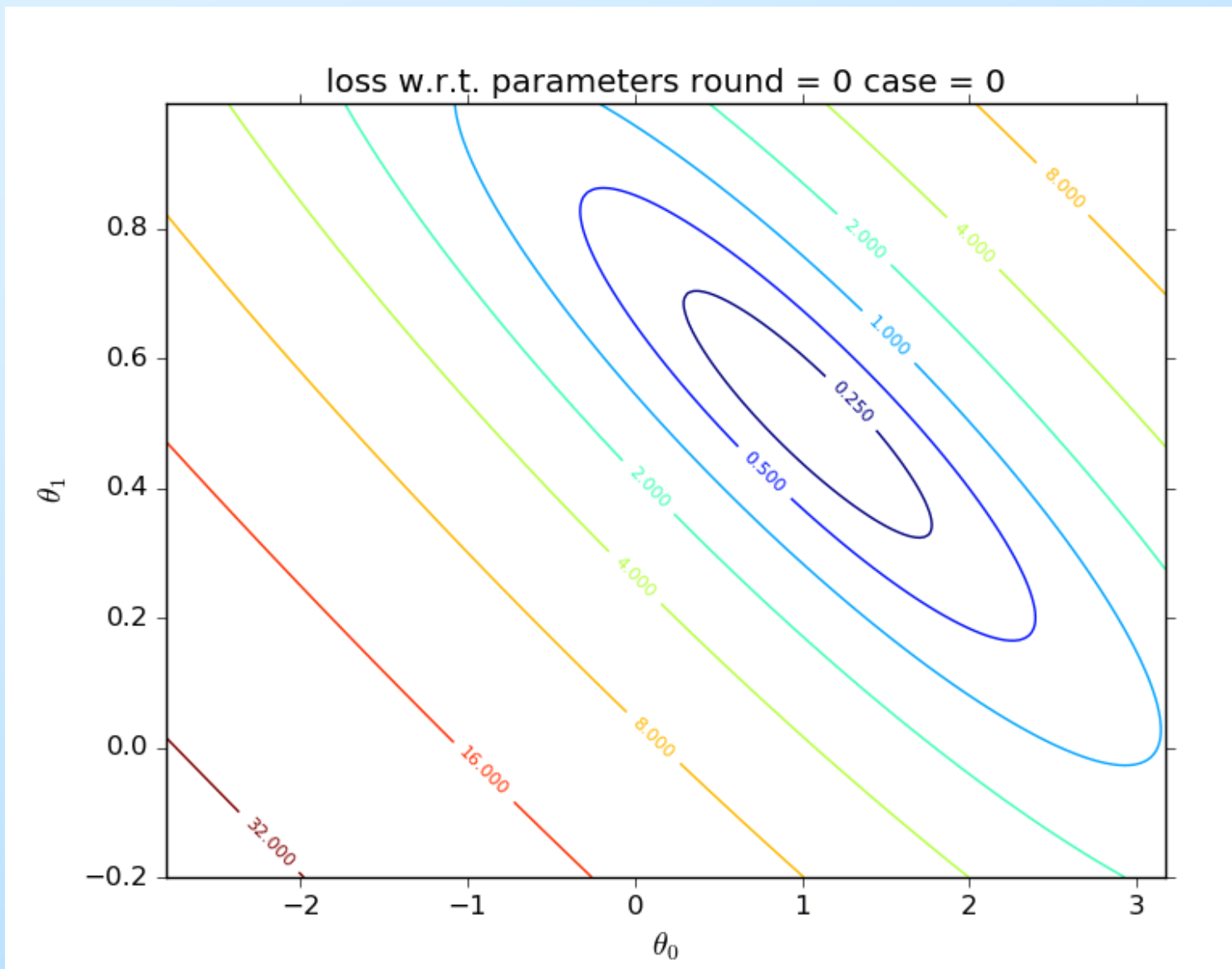
参数更新
$$\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \alpha \frac{\partial J^{(i)}(\theta)}{\partial \theta}$$
$$\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \alpha (h_{\theta}(x_i) - y_i) x_i$$

对比批量梯度下降

- ✓ 优点：极大降低梯度计算的复杂性和运算量
- ✓ 缺点：学习中不确定性或震荡

梯度下降法

随机梯度下降 (Stochastic Gradient Descent, SGD)



梯度下降法

小批量梯度下降 (Mini-Batch Gradient Descent, MBGD)

算法思想

- 梯度下降的每一步中，使用部分训练样本
- 结合了批量梯度下降和随机梯度下降的优点
- 批量梯度下降的优秀稳定性
- 随机梯度下降的快速更新

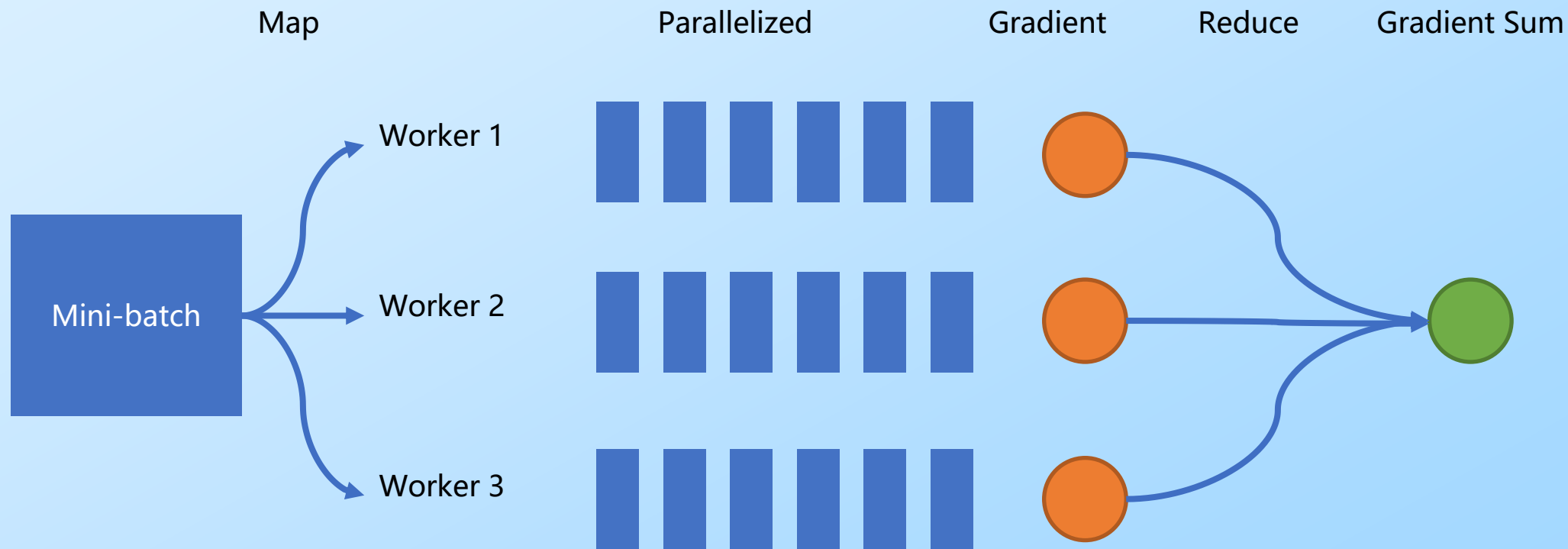
训练步骤

- 将整个训练集分成多个小批量 (mini-batches)
- 对于每一个小批量大小为 m' (远远小于 m), 做一步批量下降来降低 $J(\theta) = \frac{1}{2m'} \sum_{i=1}^{m'} (h_{\theta}(x_i) - y_i)^2$
- 对于每一个小批量, 更新参数 $\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \alpha \frac{\partial J(\theta)}{\partial \theta}$

梯度下降法

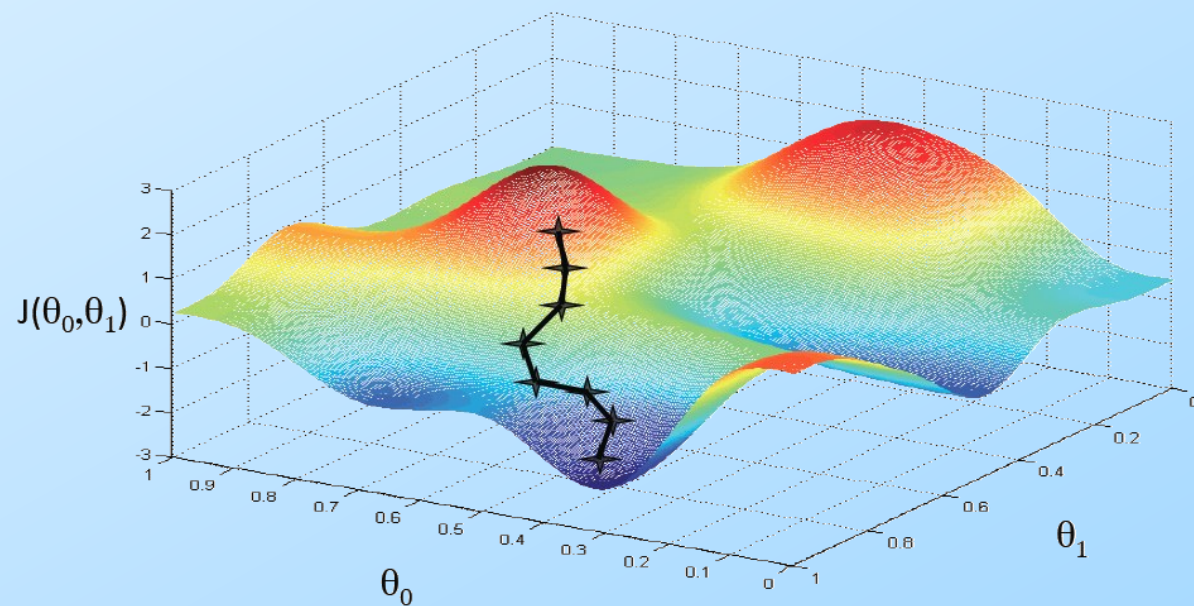
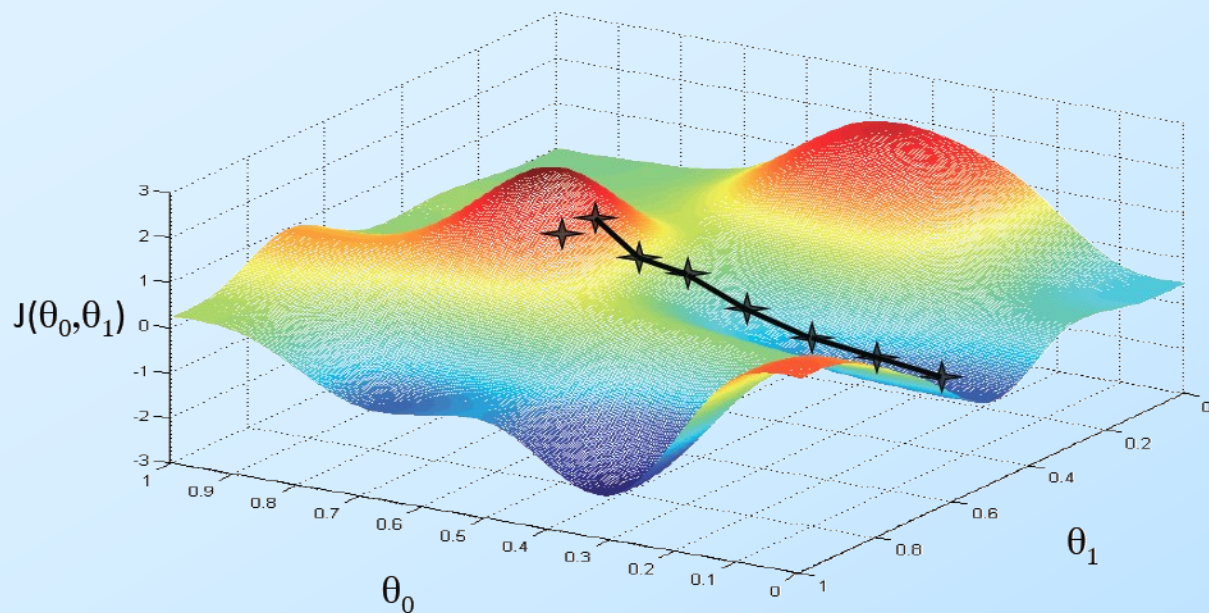
小批量梯度下降 (Mini-Batch Gradient Descent, MBGD)

- 小批量梯度下降很适合使用在并行化计算中
- 将每个小批量数据进一步切分，每个线程分别计算梯度，最后再加和这些梯度



梯度下降法

计算步骤



- ✓ 随机选择一个参数初始化 θ
- ✓ 根据数据和梯度算法来更新 θ
- ✓ 直到走到局部一个最小区域 (local minimum)

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

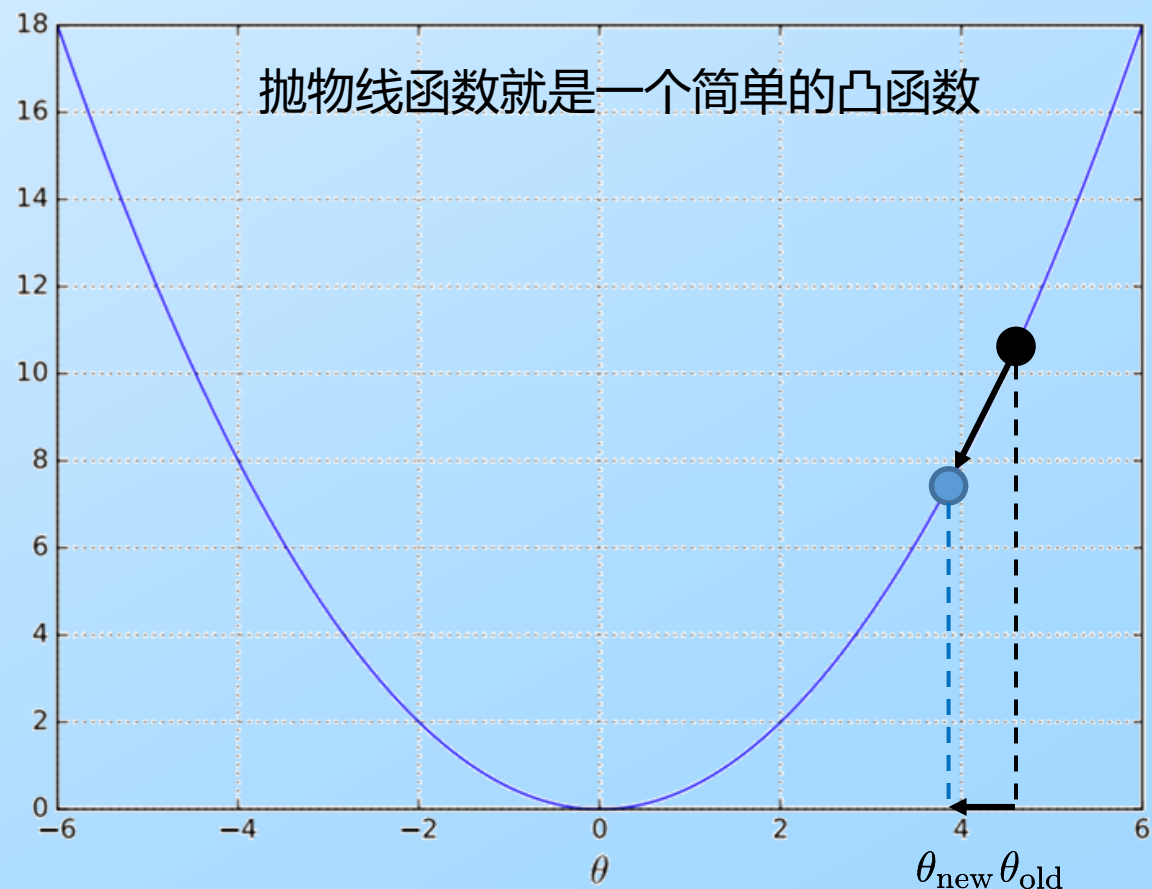
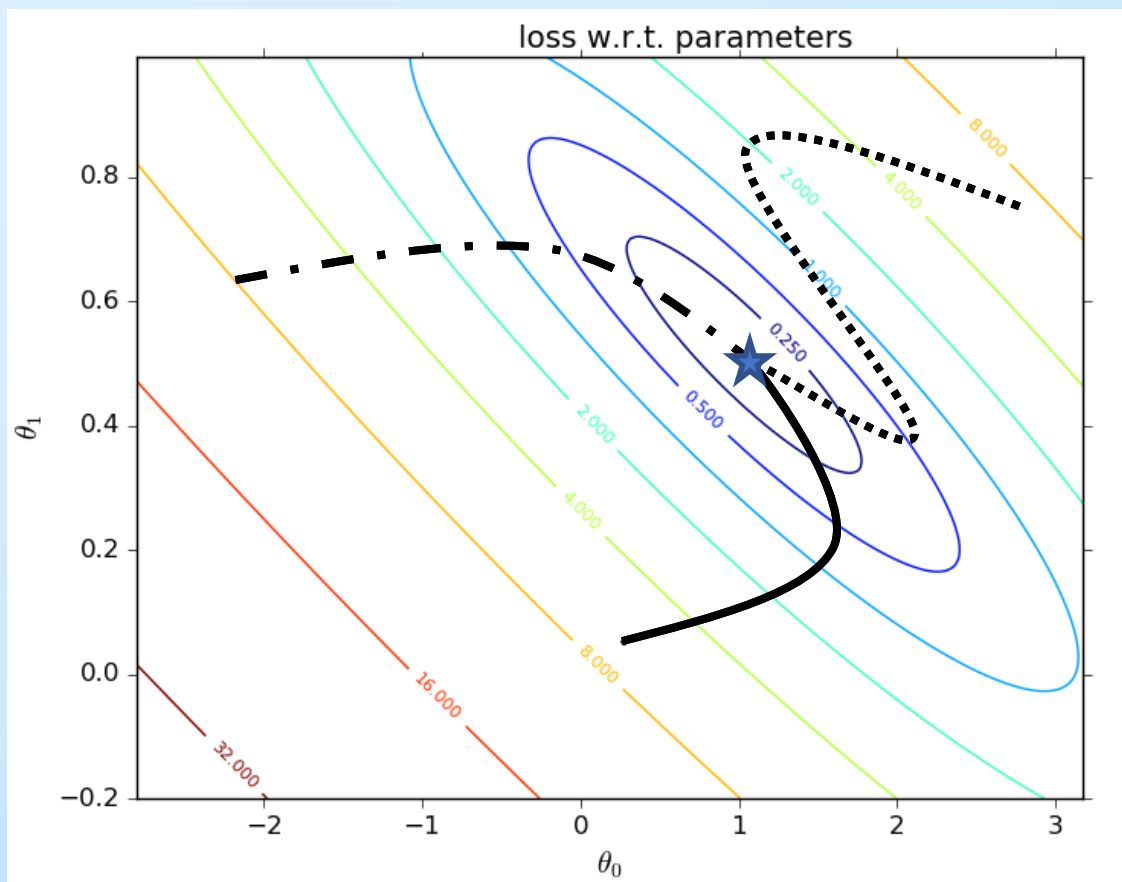
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

梯度下降法

全局最优与局部最优

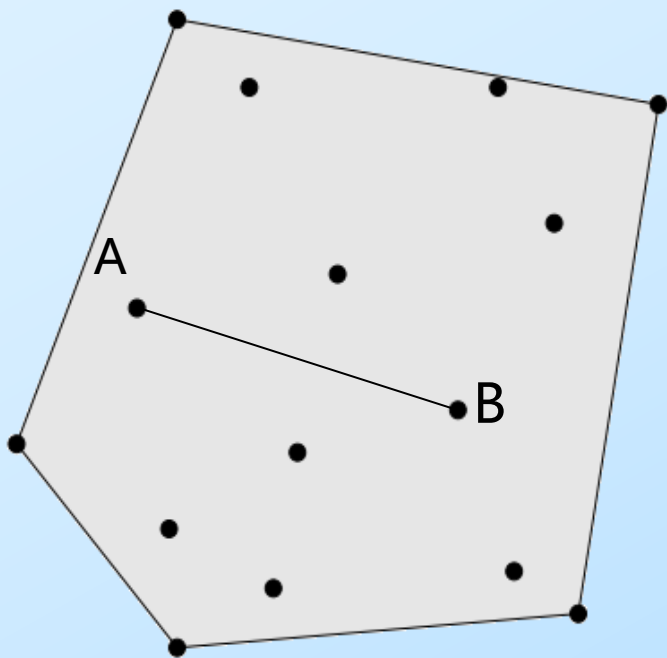
- ✓ 如果损失函数是凸函数，梯度下降法得到的解就一定是全局最优解
- ✓ 否则，梯度下降不一定能够找到全局的最优解，有可能是一个局部最优解



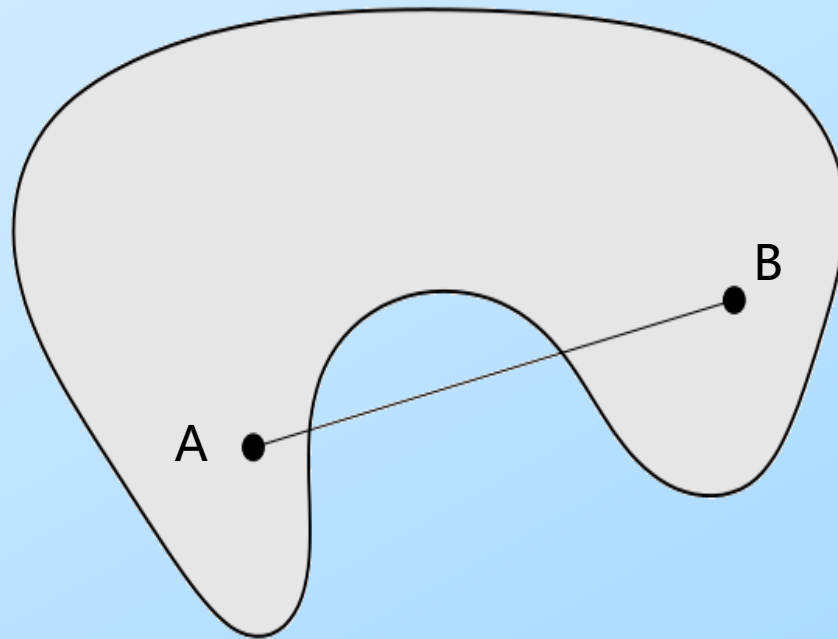
对于凸函数，不同的初始化参数最终也会学习到相同的最优值

梯度下降法

凸集



凸集



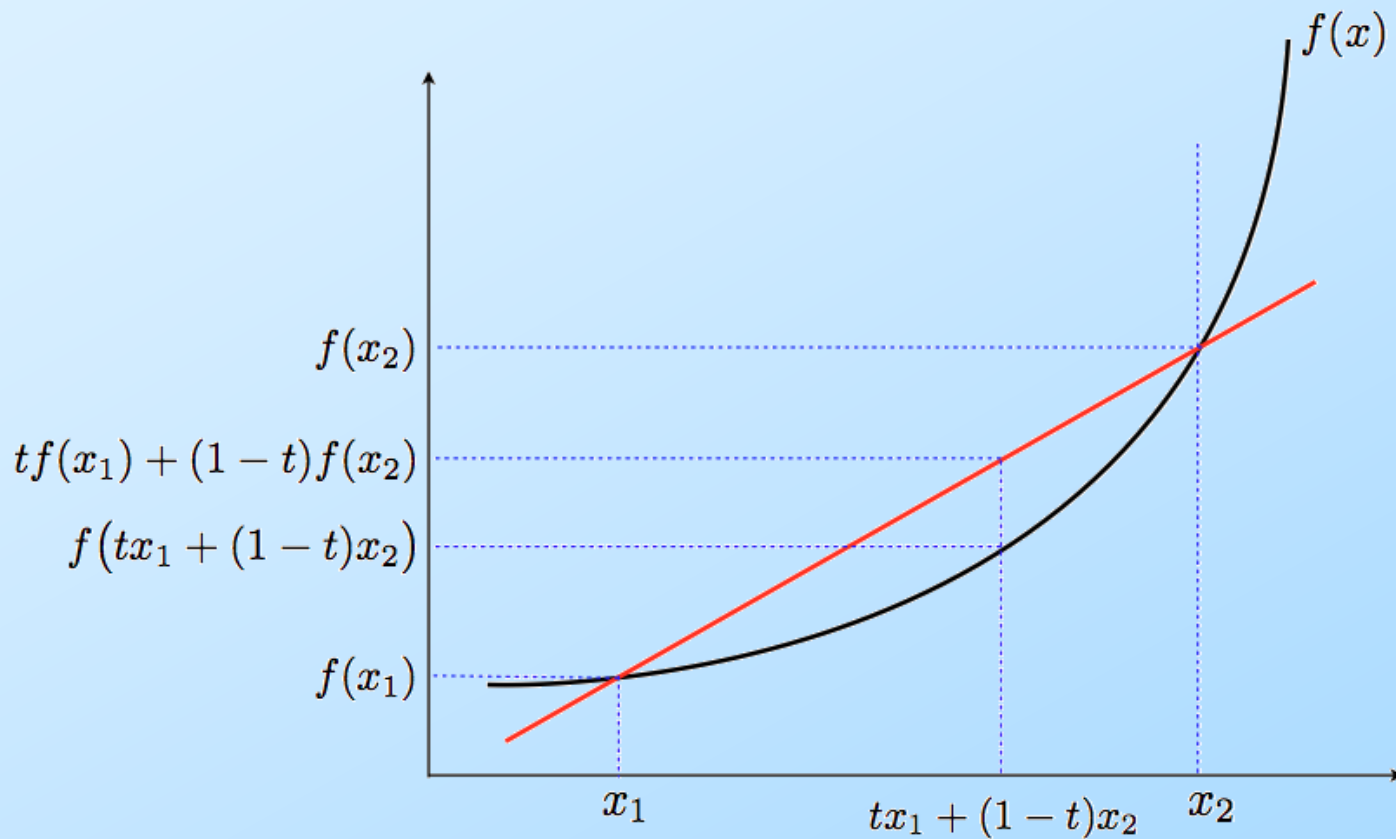
非凸集

一个点集 S 被称为凸集，当且仅当该 S 里的任意两点 A 和 B 的连线上任意一点同样属于 S

$$tx_1 + (1 - t)x_2 \in S \quad \text{for all } x_1, x_2 \in S, 0 \leq t \leq 1$$

梯度下降法

凸函数

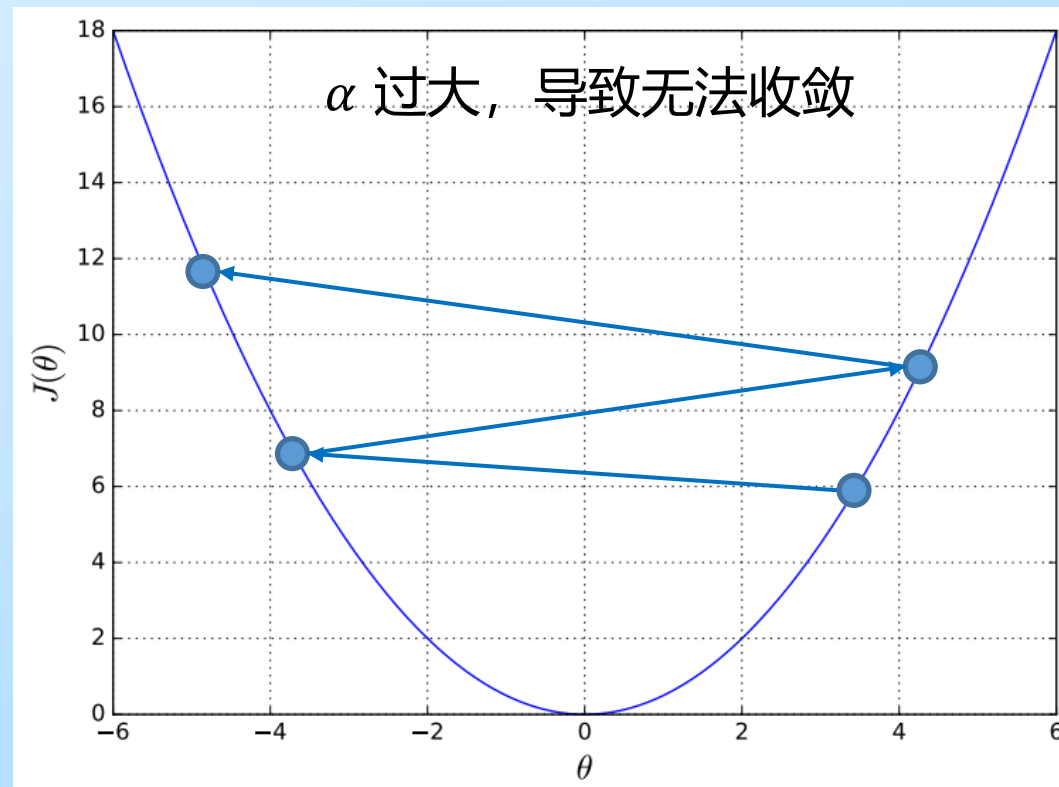
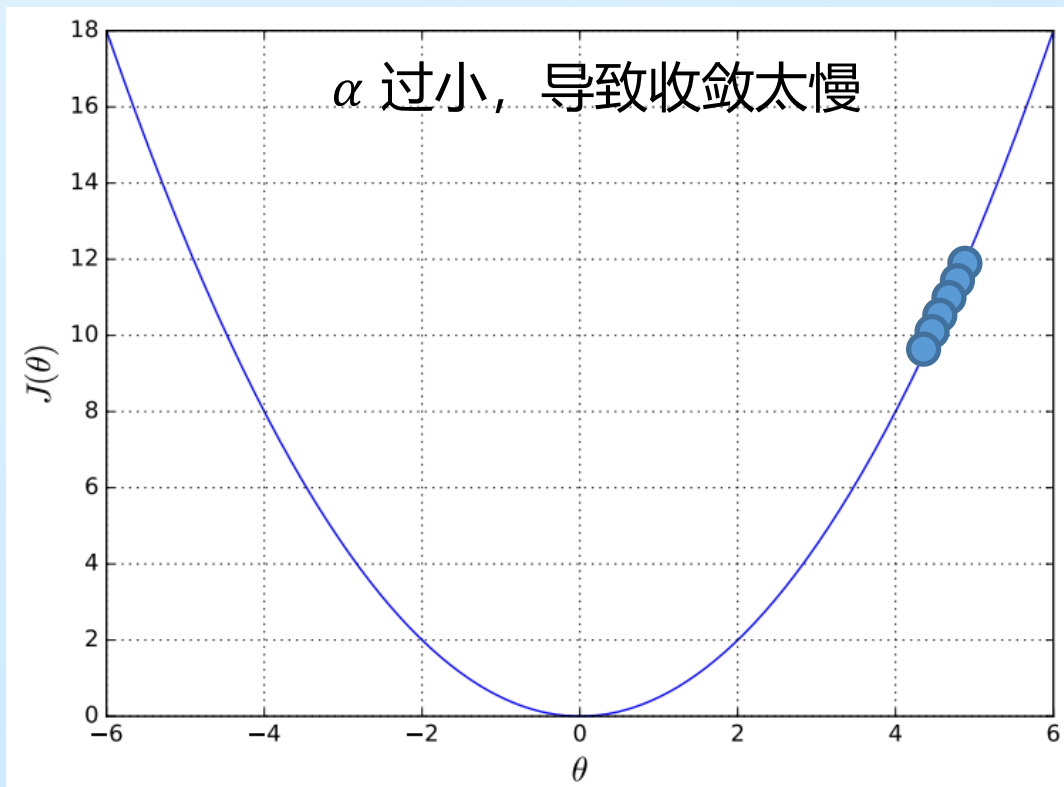


$f: \mathbb{R}^n \rightarrow \mathbb{R}$ 是凸函数: f 的定义域是一个凸集, 并且满足

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2) \quad \forall x_1, x_2 \in f \text{ 的定义域}, 0 \leq t \leq 1$$

梯度下降法

学习率的选择



$$\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

要检查梯度下降是否有效工作, 打印出损失值 $J(\theta)$ 的变化, 若 $J(\theta)$ 没有正常地下降, 调整学习率 α

梯度下降法

案例

问题

已知 $J(\theta) = \theta^2 + 3\theta + 1$, 学习率 $\alpha = 0.1$, 使用梯度下降法求解 θ 的最小值

求解

计算梯度

$$\frac{dJ(\theta)}{d\theta} = 2\theta + 3$$

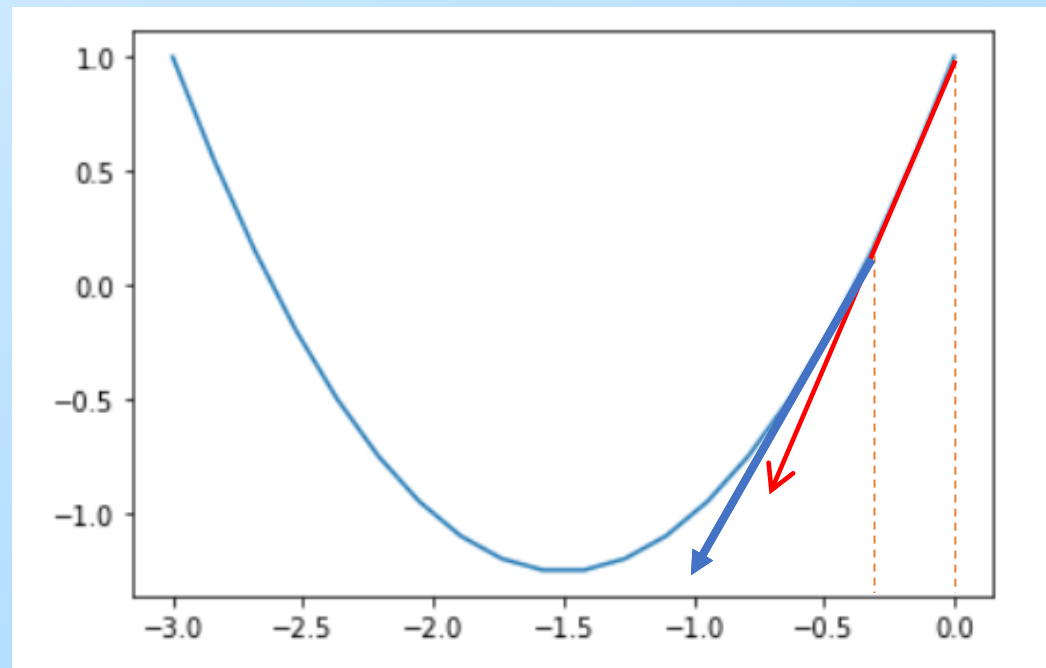
设定 θ 初始值为 $\theta^{(0)} = 0$, 运用梯度下降法可得到

$$\theta^{(1)} = \theta^{(0)} - \alpha \frac{dJ_1}{d\theta} = 0 - 0.1 * (2 * 0 + 3) = -0.3$$

$$\theta^{(2)} = \theta^{(1)} - \alpha \frac{dJ_1}{d\theta} = -0.3 - 0.1 * (2 * (-0.3) + 3) = -0.54$$

.....

依次类推, 逐步逼近最小值点-1.5



梯度下降法

数据归一化/标准化

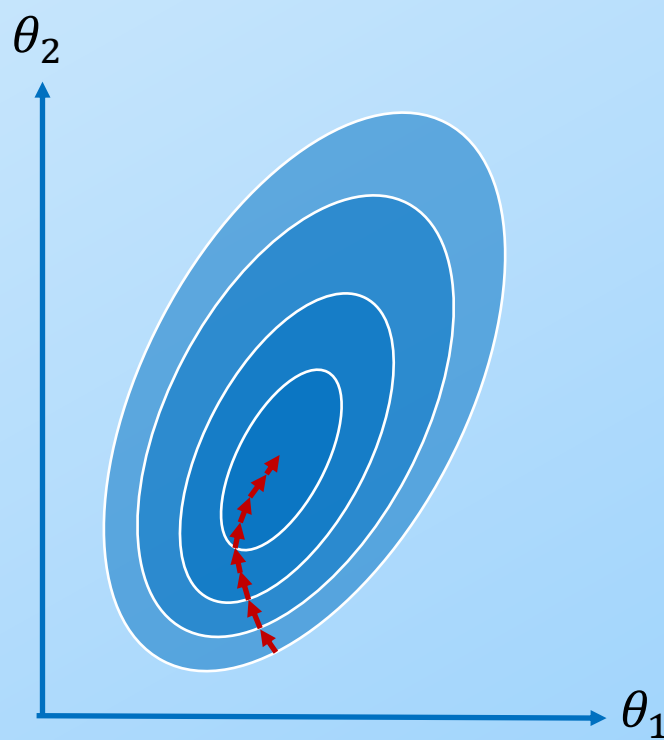
为什么要进行数据归一化/标准化?

- **提升模型精度**: 不同维度之间的特征在数值上有一定比较性, 可以大大提高分类器的准确性
- **加速模型收敛**: 最优解的寻优过程明显会变得平缓, 更容易正确的收敛到最优解

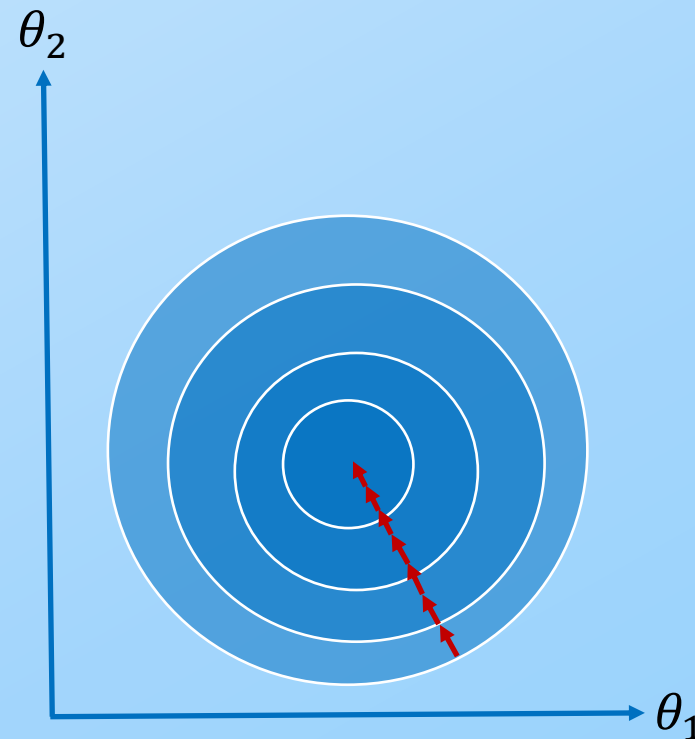
$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$x_1 = 100, \quad x_2 = 1$$

$h_{\theta}(x)$ 的值会被 x_1 主导,
 x_2 会被忽略



数据归一化前



数据归一化后

梯度下降法

数据归一化/标准化

如何进行数据归一化/标准化？

- 数据归一化：把数据缩到 [0, 1] 之间
- 数据标准化：把数据变成 均值 0，方差 1

数据归一化

$$x^* = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

数据标准化

$$x^* = \frac{x - \mu}{\sigma}$$

项目	数据归一化 Min-Max	数据标准化 Z-score
变换后范围	[0, 1]	无固定范围，均值 0 方差 1
对异常值敏感	非常敏感	相对不敏感
适用场景	神经网络像素、固定区间、距离有限制	线性回归、逻辑回归、梯度下降、PCA
计算量	小	稍大（要算均值方差）

最小二乘法

梯度下降

通过多次迭代不断逼近极小值，没有解析解，**只有数值解** $\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \alpha \frac{\partial J(\theta)}{\partial \theta}$

最小二乘

对损失函数求偏导并令其等于零 $J'(\theta) = 0$ ，所得到的 θ 即为模型参数的值

求解目标
$$\frac{\partial J(\theta)}{\partial \theta_k} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_k^{(i)} = 0$$

合并两式
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2, \quad h_{\theta}(x^{(i)}) = \theta^T x^{(i)}$$

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2$$

最小二乘法

一元线性回归

假设一元线性回归形式 $y = wx + b$, 则优化目标 $J(w, b) = \frac{1}{2m} \sum_{i=1}^m (wx_i + b - y_i)^2$

$J(w, b)$ 对 w 求导, 得到 $\frac{\partial J(w, b)}{\partial w} = \frac{1}{m} \sum_{i=1}^m (wx_i + b - y_i) * x_i = \frac{1}{m} \left(w \sum_{i=1}^m x_i^2 - \sum_{i=1}^m (y_i - b)x_i \right) = 0$

$$w = \frac{\sum_{i=1}^m (y_i - b)x_i}{\sum_{i=1}^m x_i^2}$$

$J(w, b)$ 对 b 求导, 得到 $\frac{\partial J(w, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m (wx_i + b - y_i) = b + \frac{1}{m} \sum_{i=1}^m (wx_i - y_i) = 0$

$$b = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i)$$

最小二乘法

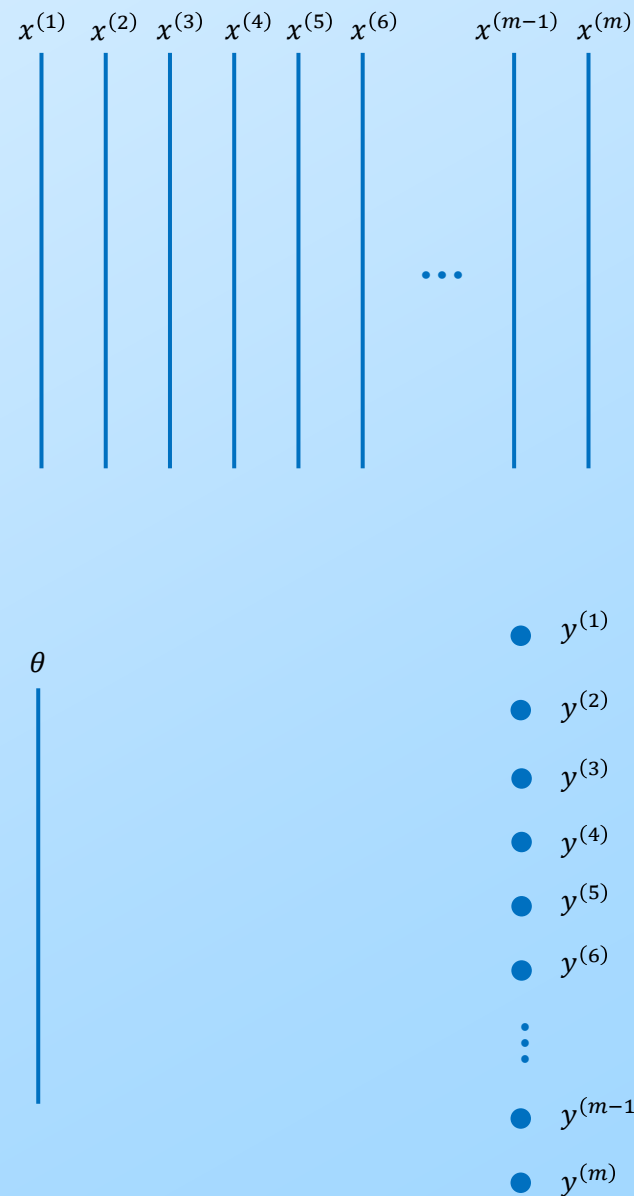
多元线性回归

求解目标 $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2$

计算每一个 loss

$$\begin{aligned} & \left[(x^{(1)})^T \theta - y^{(1)} \right]^2 \\ & \left[(x^{(2)})^T \theta - y^{(2)} \right]^2 \\ & \left[(x^{(3)})^T \theta - y^{(3)} \right]^2 \\ & \vdots \\ & \left[(x^{(m)})^T \theta - y^{(m)} \right]^2 \end{aligned}$$

写成矩阵形式 $J(\theta) = \|X^T \theta - Y\|^2$



最小二乘法

多元线性回归

求解目标 $J(\theta) = ||X^T \theta - Y||^2$

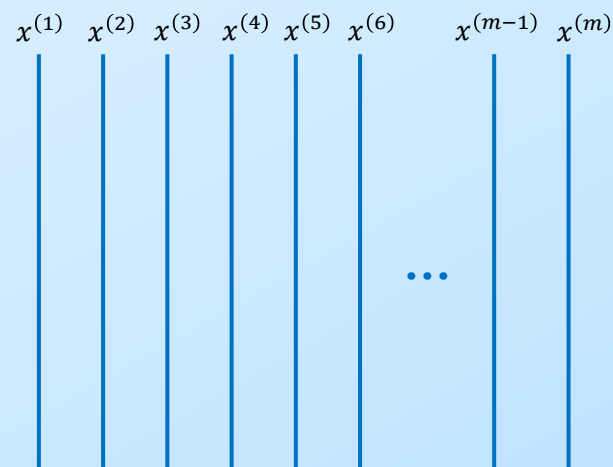
$$J(\theta) = (X^T \theta - Y)^T (X^T \theta - Y)$$

$$J(\theta) = (\theta^T X - Y^T)(X^T \theta - Y)$$

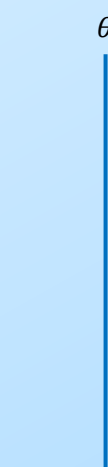
$$J(\theta) = \theta^T X X^T \theta - Y^T X^T \theta - \theta^T X Y + Y^T Y$$

$$J(\theta) = \theta^T X X^T \theta - 2\theta^T X Y + Y^T Y$$

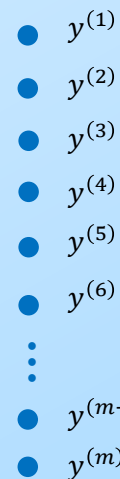
对 θ 求导 $\frac{\partial J(\theta)}{\partial \theta} = 2X X^T \theta - 2X Y$



$X \in \mathbb{R}^{d \times m}$



$\theta \in \mathbb{R}^{d \times 1}$



$Y \in \mathbb{R}^{m \times 1}$

$$\left\{ \begin{array}{l} \frac{\partial x^T A}{\partial x} = A \\ \frac{\partial x^T A x}{\partial x} = (A + A^T)x \end{array} \right.$$

最小二乘法

多元线性回归

求解目标 $\frac{\partial J(\theta)}{\partial \theta} = 2XX^T\theta - 2XY = 0$

化简得 $XX^T\theta = XY$

$$\theta = (XX^T)^{-1}XY$$

如果有新的 x^* , 则 $y = h_{\theta}(x^*) = \theta^T x^*$

梯度下降法 v.s.最小二乘法

比较	梯度下降法	最小二乘法
特征预处理	1，须要归一化处理以及选取学习速率 2，需多次迭代更新来求得最终结果	不需要
算法耗时	耗时相对较小	须要求解 $(X^T X)^{-1}$ ，其计算量大约为 $o(n^3)$ ，当训练数据集过于庞大的话，其求解过程非常耗时
如何选择	对于更复杂的学习算法或者更庞大的训练数据集，用梯度下降法较好	当模型相对简单，训练数据集相对较小，用最小二乘法较好
线性回归	当特征变量大于 10^4 时，则应该使用梯度下降法来降低计算量	当特征变量小于 10^4 时，使用最小二乘法较稳妥
适用范围	适用于各种类型的模型	只适用于线性模型，不适合逻辑回归模型等其他模型

线性回归

案例分析

具体问题

假设有一个房屋销售的数据如下：（面积，售价）

(123, 250)、(150, 320)、(87, 160)、(102, 220)

如果来了一个新的面积，假设在销售价格的记录中没有，如何预测售价？

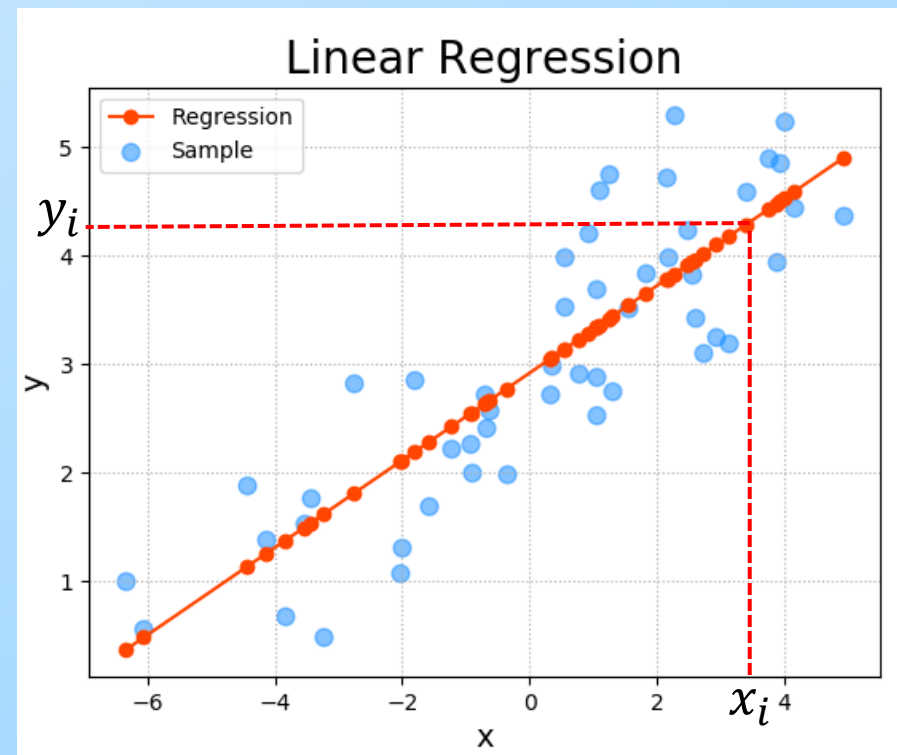
解决思路

用一条直线去拟合这些数据，当输入新的面积时，将曲线上这个点对应的值返回

$$y_i = h(x_i) = wx_i + b$$

输入面积 x_i

输出售价 y_i



线性回归

案例分析

梯度下降法

已有信息

4组数据 (123, 250) , (150, 320) , (87, 160) , (102, 220)

使用一元线性回归, 假设 $y = wx + b$, 求解 w 和 b 使 $J(w, b) = \frac{1}{2} \sum_{i=1}^m (wx_i + b - y_i)^2$ 最小化

解决方案

代入4组数据得到损失函数

$$J(w, b) = \frac{1}{2} [(w * 123 + b - 250)^2 + (w * 150 + b - 320)^2 + (w * 87 + b - 160)^2 + (w * 102 + b - 220)^2]$$

$$\frac{\partial J(w, b)}{\partial w} = \frac{1}{m} \left(w \sum_{i=1}^m x_i^2 - \sum_{i=1}^m (y_i - b)x_i \right)$$

$$= \frac{1}{4} [w(123^2 + 150^2 + 87^2 + 102^2) - ((250 - b) * 123 + (320 - b) * 150 + (160 - b) * 87 + (220 - b) * 102)]$$

$$= 13900.5w + 115.5b - 28777.5$$

线性回归

案例分析

梯度下降法

$$\begin{aligned}\frac{\partial J(w, b)}{\partial b} &= b + \frac{1}{m} \sum_{i=1}^m (wx_i - y_i) = b - \frac{1}{4} (250 - 123w + 320 - 150w + 160 - 87w + 220 - 102w) \\ &= 115.5w + b - 237.5\end{aligned}$$

给定 w 和 b 的初始值，再按照梯度下降的方法迭代更新 w 和 b

$$w^{(k+1)} = w^{(k)} - \alpha \frac{\partial J(w, b)}{\partial w}, \quad b^{(k+1)} = b^{(k)} - \alpha \frac{\partial J(w, b)}{\partial b}$$

当达到终止条件（比如达到最大迭代次数或者 w 和 b 下降幅度太小），则终止迭代，得到 w 和 b 的最终解

对于新数据 x_i ，其预测值 $y_i = wx_i + b$

线性回归

案例分析

最小二乘法

4组数据 (123, 250) , (150, 320) , (87, 160) , (102, 220)

已知 $\begin{pmatrix} w \\ b \end{pmatrix} = (XX^T)^{-1}XY$

其中 $X = \begin{pmatrix} 123 & 150 & 87 & 102 \\ 1 & 1 & 1 & 1 \end{pmatrix}$, $Y = \begin{pmatrix} 250 \\ 320 \\ 160 \\ 220 \end{pmatrix}$, 带入计算即可得到 w 和 b

对于新数据 x_i , 其预测值 $y_i = wx_i + b$

正则化约束

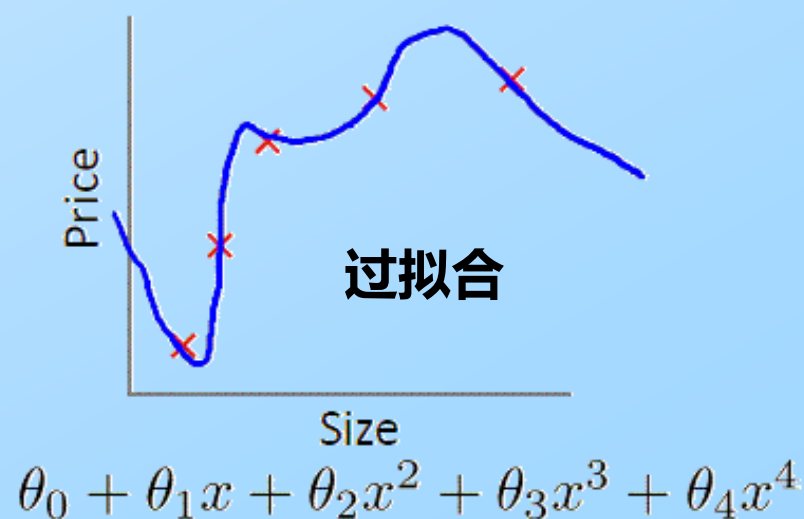
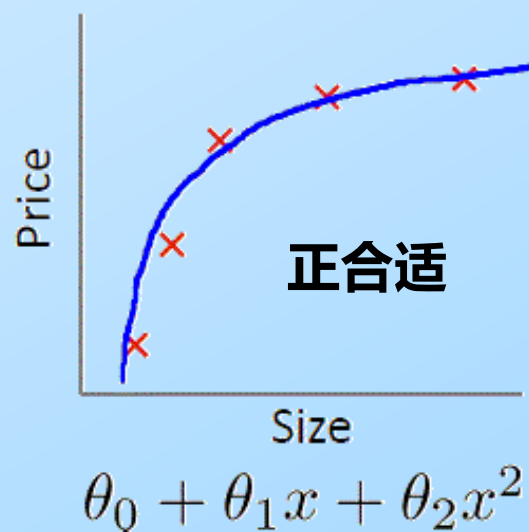
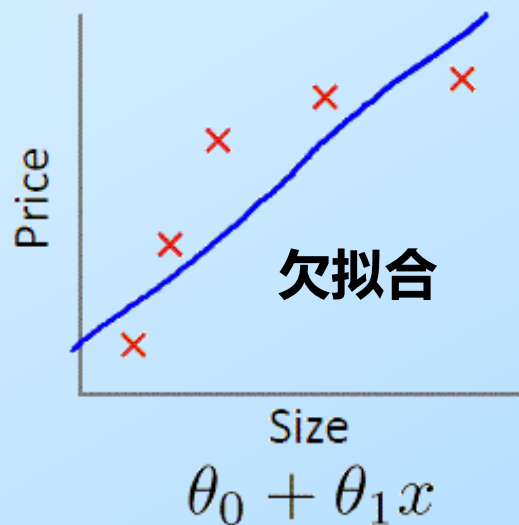
过拟合 & 欠拟合

欠拟合 (underfitting)

建立模型时，若没有选择正确的特征或者模型过于简单，使得对训练数据集的拟合建模无法精确，那么便会发生欠拟合的现象，也被称为高偏差 (bias)

过拟合 (overfitting)

建立模型时，当训练数据不够，或者模型过度/过于复杂时，便会发生过拟合的现象，也被称作高方差 (variance)



解决方案

欠拟合

添加新特征

特征选择不合理

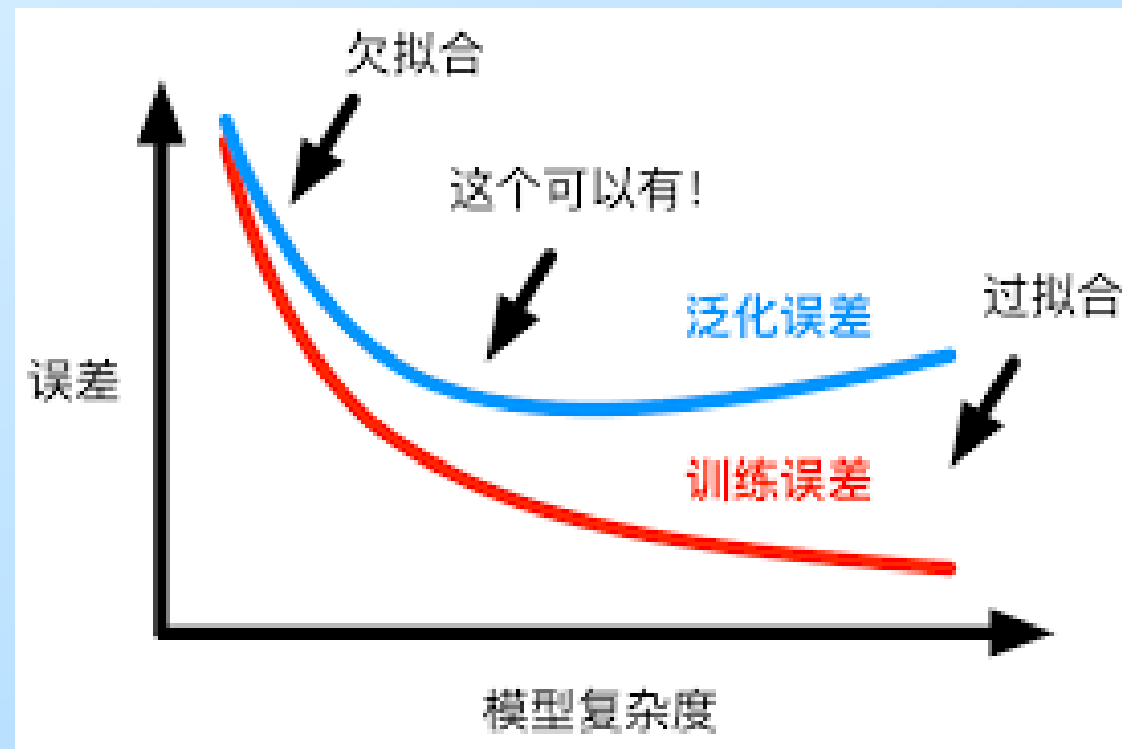
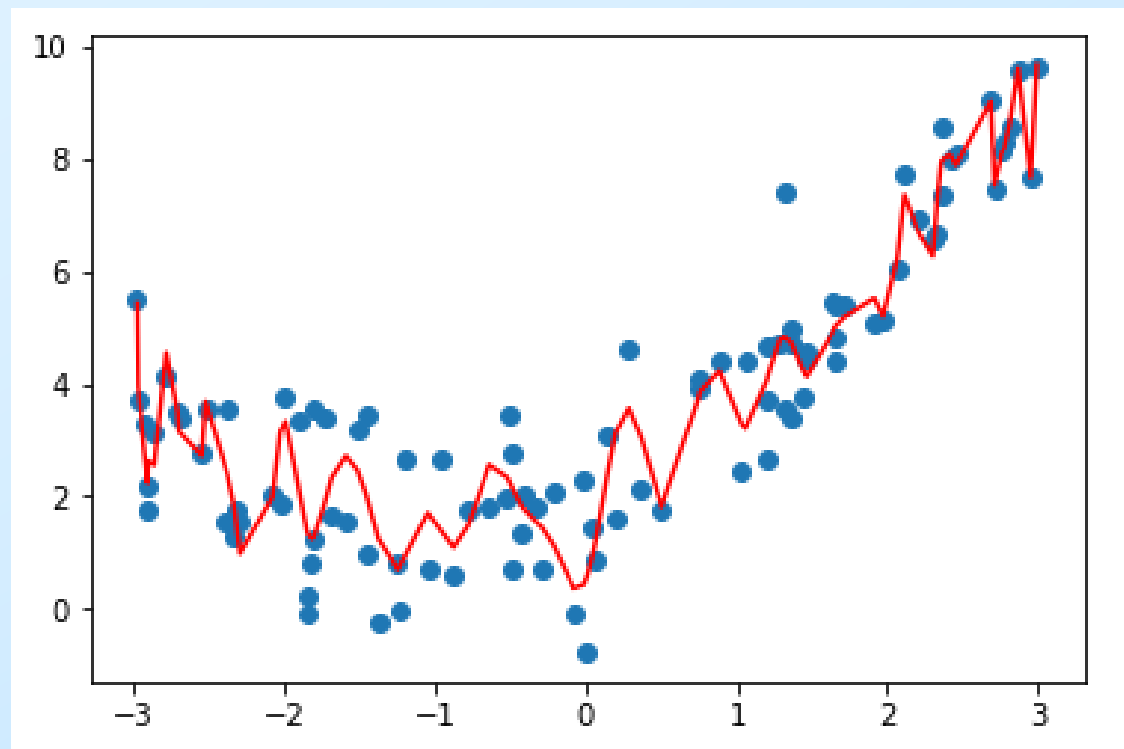
当特征不足或者现有特征与样本标签的相关性不强时，模型容易出现欠拟合。通过挖掘组合特征等新的特征，往往能够取得更好的效果

增加模型复杂度

模型设置不合理

简单模型的学习能力较差，通过增加模型的复杂度可以使模型拥有更强的拟合能力。例如，在线性模型中添加高次项，在神经网络模型中增加网络层数或神经元个数等。

过拟合



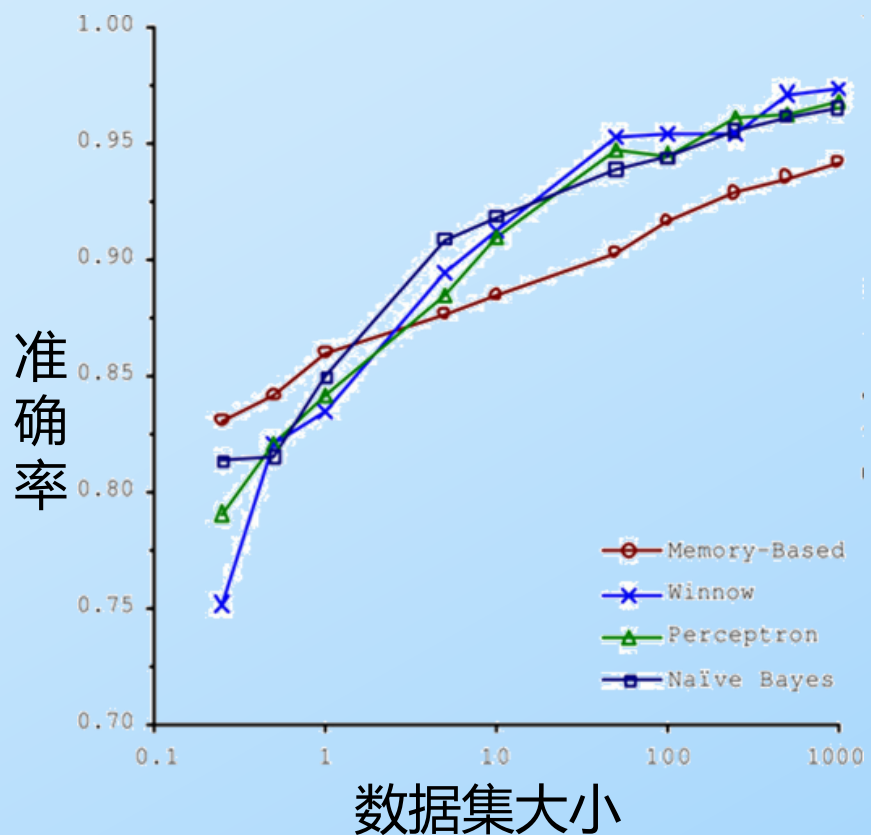
样本量过少，或者模型过于复杂

解决方案

过拟合

获得更多的训练数据

使用更多的训练数据是解决过拟合问题最有效的手段，因为更多的样本能够让模型学习到更多更有效的特征，减小噪声的影响



通过这张图可以看出，各种不同算法在输入的数据量达到一定级数后，都有相近的高准确度。于是诞生了机器学习界的名言：

成功的机器学习应用不是拥有最好的算法，而是拥有最多的数据！

解决方案

过拟合

降维

丢弃一些不能帮助我们正确预测的特征，可以是手工选择保留哪些特征，或者使用一些模型选择的算法来帮忙（例如PCA）

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- 降低了噪声特征的干扰
- 变相的降低了参数量，精简了模型复杂度

解决方案

过拟合

正则化 (regularization)

正则化的技术，保留所有的特征，但是减少参数的大小，它可以改善或者减少过拟合问题

$$J(\theta) \leftarrow J(\theta) + \text{Penalty}(\theta)$$

L1正则化/Lasso回归: $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |\theta_j|$

L2正则化/岭回归: $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2$

λ 为正则化系数，调整正则化项与训练误差的比例， $\lambda > 0$

解决方案

过拟合

正则化

Lasso回归：参数范数为L1范数

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |\theta_j| \right]$$

优点：不仅可以解决过拟合问题，而且可以在参数缩减过程中，将一些重复或不重要的参数直接缩减为零（删除），有提取有用特征的作用。

缺点：计算过程复杂，毕竟L1范数不是连续可导的。

学习链接：<https://www.cnblogs.com/Belter/p/8536939.html>

解决方案

过拟合

正则化

岭回归/脊回归/L2正则：在对损失函数 $J(\theta)$ 进行优化的时候，在其后插入一正则项，其中一种方法是“岭回归”，其损失函数正则化后如下：

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

其中 λ 为正则化参数，其作用是控制**拟合训练数据的目标**和**保持参数值较小的目标**之间的平衡关系，这种正则化线性回归称作“**岭回归**”

$$\min J(\theta) = \min \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right], \text{ 求偏导并令其等于0 } \Rightarrow \theta = (XX^T + \lambda I)^{-1}XY$$

解决方案

过拟合

正则化

求解目标

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$$J(\theta) = ||X^T \theta - Y||^2 + \lambda ||\theta||^2$$

$$J(\theta) = \theta^T X X^T \theta - 2\theta^T X Y + Y^T Y + \lambda \theta^T \theta$$

$$J(\theta) = \theta^T X X^T \theta - 2\theta^T X Y + Y^T Y + \lambda \theta^T \theta$$

对 θ 求导

$$\frac{\partial J(\theta)}{\partial \theta} = 2(X X^T + \lambda I)\theta - 2X Y = 0$$

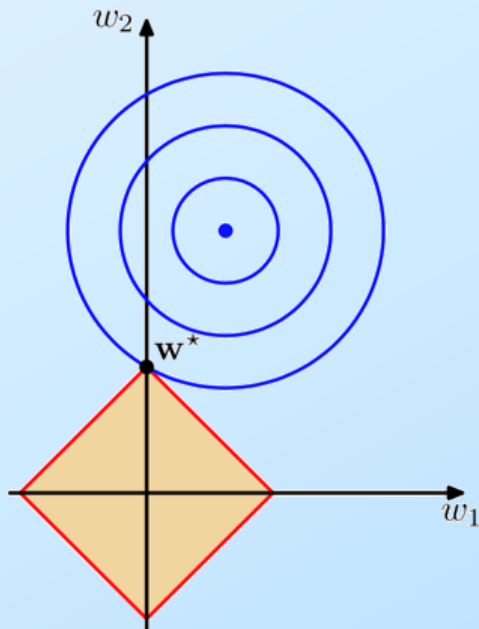
$$\theta = (X X^T + \lambda I)^{-1} X Y$$

解决方案

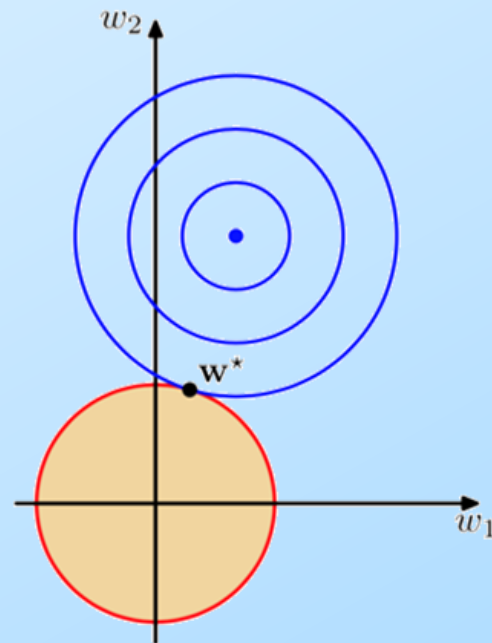
过拟合

正则化

L1正则化是指在损失函数中加入权值向量 w 的绝对值之和，L1的功能是使权重稀疏



L1正则化可以产生稀疏模型

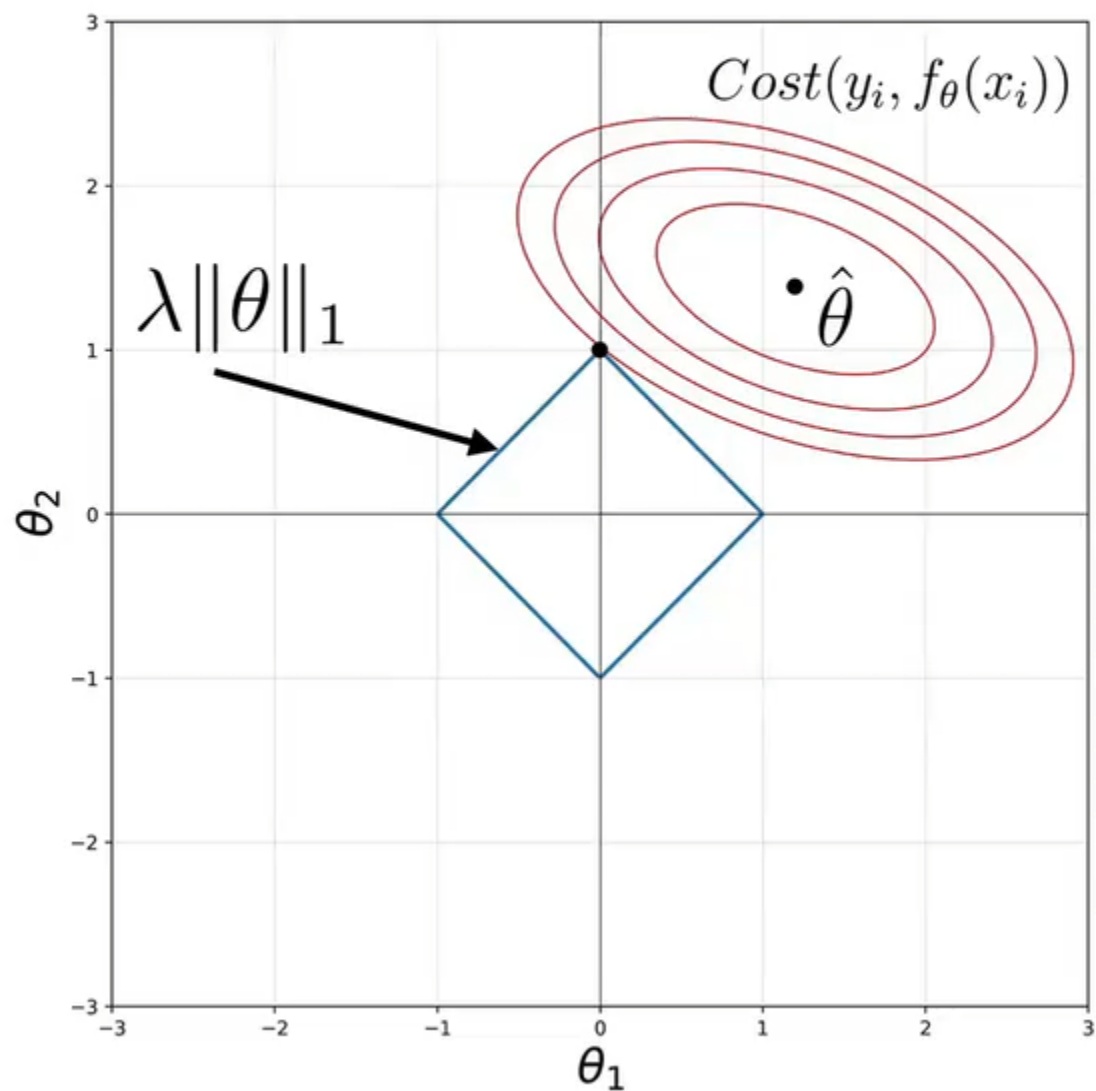


L2正则化可以防止过拟合

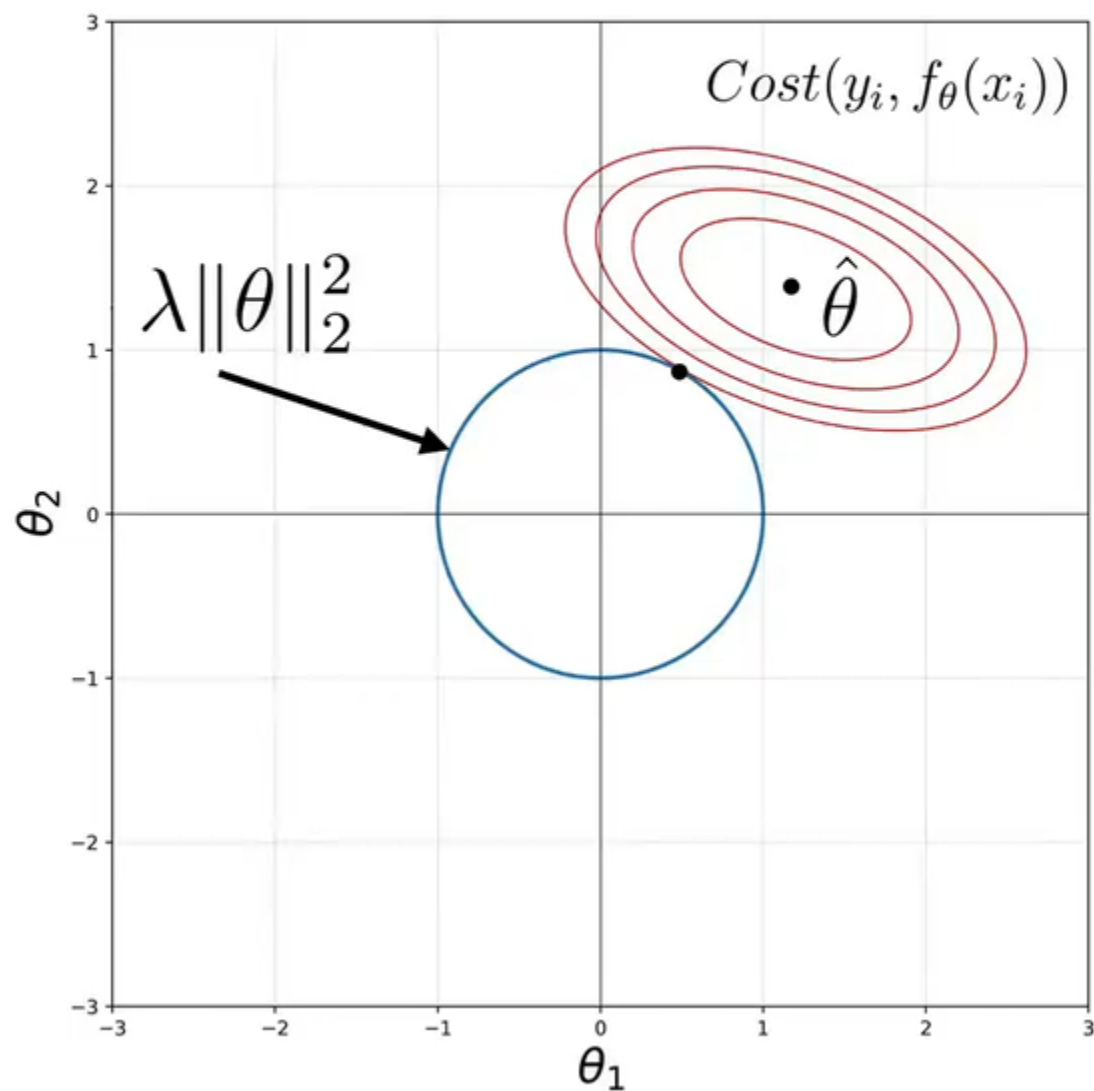
在损失函数中加入权值向量 w 的平方和，L2的功能是使权重平滑

- 蓝色轮廓线是没有正则化损失函数的等高线，中心的蓝色点为最优解
- L1正则化的最优解 w^* 是使解更加靠近某些轴,而其它的轴则为0,所以L1正则化能使得到的参数稀疏化
- L2正则化的最优解 w^* 是使解更加靠近原点,也就是说 L2正则化能降低参数范数的总和

L1正则化



L2正则化

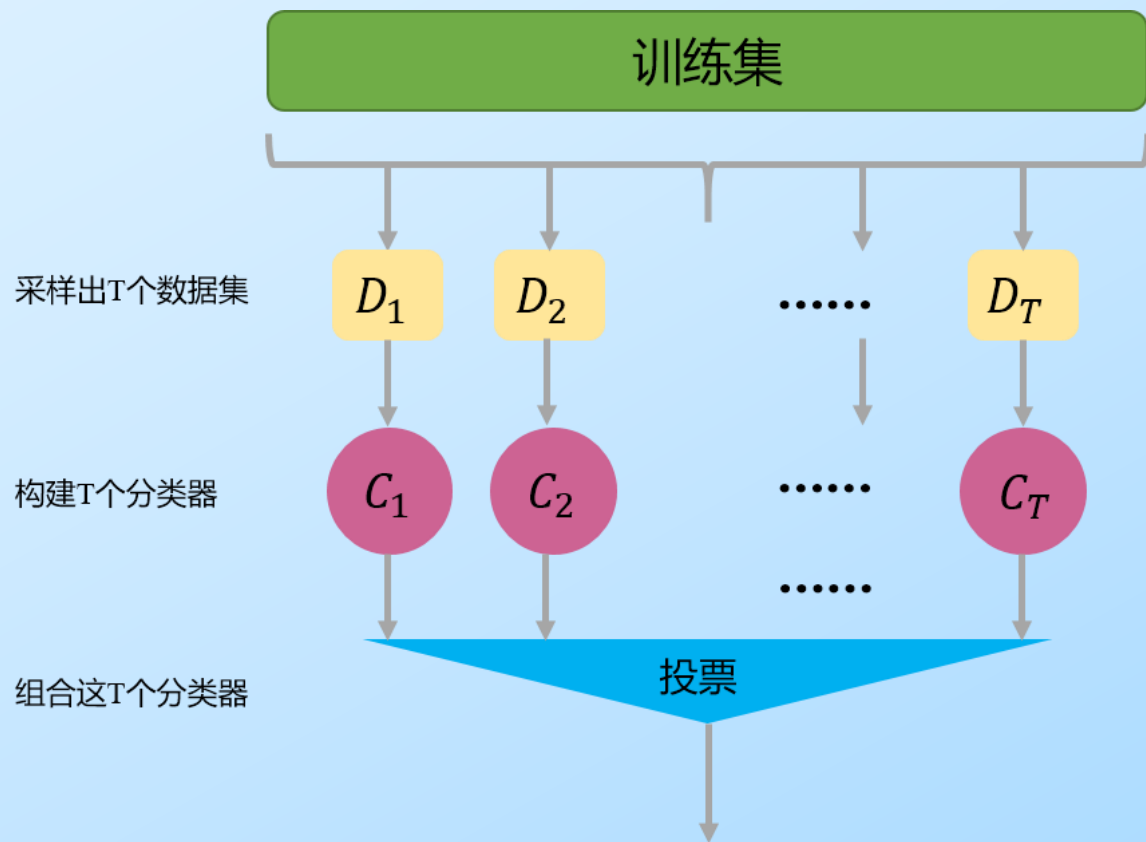


解决方案

过拟合

集成学习方法

集成学习是把多个模型集成在一起，来降低单一模型的过拟合风险



评价指标

均方误差 (Mean Square Error, MSE)

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

平均绝对误差 (Mean Absolute Error, MAE)

$$\text{MAE}(y, \hat{y}) = \frac{1}{m} \sum_{i=1}^m |y^{(i)} - \hat{y}^{(i)}|$$

均方根误差 RMSE(Root Mean Square Error, RMSE)

$$\text{RMSE}(y, \hat{y}) = \sqrt{\frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2}$$

其中, $y^{(i)}$ 和 $\hat{y}^{(i)}$ 分别表示第 i 个样本的真实值和预测值, m 为样本个数

线性回归原理总结

- 线性回归模型: $h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$
- 目标函数: $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$
- 参数估计方法: 最小二乘法, 梯度下降法
- 遇到的问题: 过拟合
- 解决方法: 引入正则项